

BỘ GIÁO DỤC VÀ ĐÀO TẠO BỘ NÔNG NGHIỆP VÀ MÔI TRƯỜNG
PHÂN HIỆU TRƯỜNG ĐẠI HỌC THỦY LỢI



NGUYỄN THÀNH TÂM

TÊN ĐỀ TÀI
XÂY DỰNG HỆ THỐNG GIÁM SÁT GIAO THÔNG THÔNG
MINH PHÁT HIỆN, THEO DÕI VÀ PHÂN TÍCH HÀNH VI VI
PHẠM PHƯƠNG TIỆN DỰA TRÊN NỀN TẢNG DEEP
LEARNING

ĐỒ ÁN TỐT NGHIỆP

TP. HỒ CHÍ MINH, THÁNG 6 NĂM 2026

BỘ GIÁO DỤC VÀ ĐÀO TẠO BỘ NÔNG NGHIỆP VÀ MÔI TRƯỜNG
PHÂN HIỆU TRƯỜNG ĐẠI HỌC THỦY LỢI

NGUYỄN THÀNH TÂM

TÊN ĐỀ TÀI
XÂY DỰNG HỆ THỐNG GIÁM SÁT GIAO THÔNG THÔNG
MINH PHÁT HIỆN, THEO DÕI VÀ PHÂN TÍCH HÀNH VI VI
PHẠM PHƯƠNG TIỆN DỰA TRÊN NỀN TẢNG DEEP
LEARNING

Ngành: Công nghệ thông tin

Mã số: TLS106

NGƯỜI HƯỚNG DẪN: TS. NGUYỄN VĂN HÁN

TP. HỒ CHÍ MINH, THÁNG 6 NĂM 2026

GÁY BÌA ĐỒ ÁN TỐT NGHIỆP

NGUYỄN THÀNH TÂM

ĐỒ ÁN TỐT NGHIỆP

HỒ CHÍ MINH, NĂM 2026

LỜI CAM ĐOAN

Tôi xin cam đoan Đồ án tốt nghiệp với đề tài "Xây dựng hệ thống Giám sát giao thông thông minh phát hiện, theo dõi và phân tích hành vi vi phạm phương tiện dựa trên nền tảng Deep Learning" là công trình nghiên cứu do chính tôi thực hiện dưới sự hướng dẫn của TS. Nguyễn Văn Hán.

Toàn bộ nội dung, số liệu, giải thuật và kết quả triển khai thực nghiệm được trình bày trong báo cáo này là hoàn toàn trung thực, mang tính nguyên bản và chưa từng được công bố trong bất kỳ công trình nào khác. Các thông tin, tài liệu, mã nguồn mở và kiến thức tham khảo từ các tác giả khác đều được tôi chú thích, trích dẫn nguồn gốc rõ ràng, minh bạch.

Tôi xin chịu hoàn toàn trách nhiệm trước Hội đồng đánh giá Đồ án tốt nghiệp và Phân hiệu Trường Đại học Thủy Lợi về những lời cam đoan trên.

Tác giả ĐATN

(Ký và ghi rõ họ tên)

Nguyễn Thành Tâm

LỜI CẢM ƠN

Em xin gửi lời tri ân sâu sắc đến Quý Thầy Cô bộ môn Công nghệ thông tin - Phân hiệu Trường Đại học Thủy Lợi. Cảm ơn Thầy Cô đã tận tâm giảng dạy, truyền đạt những kiến thức nền tảng vững chắc và định hướng nghề nghiệp cho em từ những ngày đầu bỡ ngỡ bước vào trường.

Đặc biệt, em xin bày tỏ lòng biết ơn chân thành nhất đến TS.Nguyễn Văn Hán. Thầy đã trực tiếp hướng dẫn, định hướng nghiên cứu và chỉ bảo em vô cùng tận tình trong suốt quá trình thực hiện đồ án.

Bên cạnh đó, em cũng xin gửi lời cảm ơn sâu sắc đến gia đình, bố mẹ – những người luôn là hậu phương vững chắc, hy sinh và động viên em trong những giai đoạn áp lực nhất của chặng đường học tập.

Mặc dù em đã nỗ lực hết mình để nghiên cứu và phát triển sản phẩm, song do giới hạn về mặt thời gian cũng như kinh nghiệm thực tiễn, đồ án chắc chắn không tránh khỏi những thiếu sót. Em rất mong nhận được sự đánh giá, góp ý từ Quý Thầy Cô trong Hội đồng bảo vệ để đề tài được hoàn thiện hơn và có cơ hội ứng dụng vào thực tiễn.

Em xin chân thành cảm ơn!

MỤC LỤC

LỜI CAM ĐOAN	ii
LỜI CẢM ƠN.....	iii
MỤC LỤC	iv
DANH MỤC CÁC HÌNH ẢNH.....	vii
DANH MỤC BẢNG BIỂU.....	ix
DANH MỤC CÁC TỪ VIẾT TẮT VÀ GIẢI THÍCH CÁC THUẬT NGỮ	x
CHƯƠNG 1 TỔNG QUAN ĐỀ TÀI VÀ CƠ SỞ LÝ THUYẾT DEEP LEARNING...1	
1.1 Đặt vấn đề và Mục tiêu nghiên cứu	1
1.1.1 Đặt vấn đề.....	1
1.1.2 Mục tiêu nghiên cứu.....	2
1.2 Cơ sở lý thuyết về Deep Learning trong nhận diện hình ảnh.....	3
1.2.1 Mạng nơ-ron tích chập	3
1.2.2 Bài toán Phát hiện đối tượng.....	4
1.3 Kiến trúc mạng YOLOv8	6
1.3.1 Lịch sử phát triển của họ YOLO	6
1.3.2 Cấu trúc cốt lõi của YOLOv8	6
1.3.3 Lý do lựa chọn phiên bản YOLOv8 Small.....	8
1.4 Bài toán Theo dõi đa đối tượng	10
1.4.1 Khái niệm và những thách thức trong bài toán theo dõi đa đối tượng....	10
1.4.2 Giải thuật Tracking ByteTrack.....	11
1.5 Cơ sở lý thuyết về Nhận dạng ký tự quang học.....	13
1.6 Các nền tảng hỗ trợ đóng gói hệ thống.....	13
CHƯƠNG 2 :XÂY DỰNG, HUẤN LUYỆN MÔ HÌNH VÀ THIẾT KẾ HỆ THỐNG	16
2.1 Kiến trúc luồng xử lý dữ liệu AI.....	16
2.1.1 Luồng tuần tự của hệ thống nhận diện và phân tích hành vi.....	16
2.1.2 Sơ đồ khối tổng thể của hệ thống	17
2.2 Huấn luyện và Tối ưu mô hình YOLOv8	18

2.2.1	Chuẩn bị và tiền xử lý tập dữ liệu	18
2.2.2	Kỹ thuật tăng cường dữ liệu và cấu hình siêu tham số	19
2.2.3	Đánh giá kết quả huấn luyện	21
2.3	Giải thuật hình học phân tích vùng quan tâm	23
2.3.1	Bản chất toán học của việc thiết lập vùng đa giác kiểm soát	23
2.3.2	Thuật toán kiểm tra điểm nằm trong đa giác	24
2.4	Đề xuất Logic và Giải thuật phát hiện vi phạm	25
2.4.1	Giải thuật đo tốc độ phương tiện dựa trên phép biến đổi phối cảnh	25
2.4.2	Giải thuật phát hiện phương tiện đi ngược chiều	26
2.4.3	Giải thuật phát hiện phương tiện vượt đèn đỏ	27
2.5	Giải thuật trích xuất và chuẩn hóa dữ liệu biển số xe	28
2.5.1	Tiền xử lý hình ảnh vùng biển số	28
2.5.2.	Logic chuẩn hóa format biển số Việt Nam	29
2.6	Tích hợp phân tích luồng lưu lượng API và Quản trị Cơ sở dữ liệu	30
2.6.1	Cấu trúc cơ sở dữ liệu lưu trữ hành vi vi phạm	30
2.6.2	Tính toán mật độ kẹt xe dựa trên API TomTom	31
2.7	Hệ thống thông tin liên lạc và Cảnh báo tự động qua Telegram Bot	32
2.7.1	Kiến trúc giao tiếp bất đồng bộ của mạng cảnh báo	32
2.7.2	Giải thuật đóng gói nội dung và cơ chế trigger Telegram	32
2.8	Tích hợp Trợ lý ảo Trí tuệ nhân tạo hỗ trợ truy vấn thông tin	34
2.8.1	Mục đích và kiến trúc hoạt động	34
2.8.2	Thuật toán gò cấu trúc và chống ảo giác cho AI	35
CHƯƠNG 3 : TRIỂN KHAI THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ		36
3.1	Cấu hình môi trường thử nghiệm và Triển khai hệ thống	36
3.1.1	Cấu hình phần cứng và phần mềm hệ thống	36
3.1.2	Giải pháp phân tách môi trường và Cơ chế Tunneling bảo mật	38
3.2	Kết quả triển khai thực tế ứng dụng giám sát giao thông	39
3.2.1	Giao diện Giám sát trực tiếp và Vẽ vùng kiểm soát AI	39
3.2.2	Trung tâm Điều hành lưu lượng diện rộng	41
3.2.3	Trung tâm Quản trị hệ thống và Định tuyến dữ liệu	43

3.2.4	Phân hệ Định danh và Bảo mật hệ thống	44
3.2.5	Giao diện Phân tích thống kê và Lọc dữ liệu vi phạm	45
3.2.6	Giao diện Trợ lý ảo AI tương tác thời gian thực	47
3.2.7	Đánh giá thực nghiệm hiệu năng giải thuật Tracking và OCR	48
KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN		52
TÀI LIỆU THAM KHẢO		55

DANH MỤC CÁC HÌNH ẢNH

Hình 1.1 Trung tâm giám sát giao thông.....	2
Hình 1.2 So sánh kiến trúc luồng xử lý của bộ phát hiện đối tượng Two-stage và One-stage.	6
Hình 1.3 Sơ đồ kiến trúc chi tiết của mô hình mạng YOLOv8. [5]	8
Hình 1.4 Quy trình liên kết dữ liệu hai giai đoạn của giải thuật ByteTrack.	12
Hình 2.1 Sơ đồ khối tổng thể luồng kiến trúc pipeline xử lý dữ liệu AI của hệ thống.	18
Hình 2.2 Minh họa quá trình gán nhãn tọa độ khung bao cho các phương tiện giao thông.	19
Hình 2.3 Phân bố số lượng nhãn các lớp đối tượng trên nền tảng Roboflow.	19
Hình 2.4 Môi trường cấu hình và tiến trình huấn luyện mô hình YOLOv8s trên Kaggle.....	21
Hình 2.5 Các đường cong hàm mất mát (Loss) và biểu đồ chỉ số mAP đánh giá chất lượng mô hình sau huấn luyện.....	22
Hình 2.6 Ma trận nhầm lẫn (Confusion Matrix) và các biểu đồ phân tích độ chính xác trên từng lớp đối tượng.	23
Hình 2.7 Giải thuật Ray Casting.....	25
Hình 2.8 Minh họa quy trình tiền xử lý hình thái học ảnh biển số trước khi đưa qua mô hình OCR. [10]	29
Hình 2.9 Thông báo được gửi về telebot cảnh báo từ xa	34
Hình 3.1 Sơ đồ kiến trúc triển khai phân tán kết hợp kết nối đường hầm bảo mật của hệ thống.	39
Hình 3.2 Giao diện Trung tâm Giám sát trực tiếp, hỗ trợ quản lý nguồn phát video và cấu hình chức năng AI.....	41
Hình 3.3 Khung bản đồ số định vị vị trí trạm camera kết hợp trích xuất thời tiết thực tế theo thời gian thực.	41
Hình 3.4 Giao diện Bản đồ lưu lượng tại các nút giao ở TP.Hồ Chí Minh dựa trên tích hợp dữ liệu API TomTom	42
Hình 3.5 Giao diện Trung tâm Điều hành lưu lượng giao thông diện rộng dựa trên tích hợp dữ liệu API TomTom.	43
Hình 3.6 Giao diện Trung tâm Quản trị hệ thống hỗ trợ đồng bộ dữ liệu thời gian thực xuống PostgreSQL.....	44

Hình 3.7 Giao diện đăng ký/đăng nhập bảo mật hệ thống	45
Hình 3.8 Giao diện Phân tích dữ liệu hỗ trợ công cụ lọc đa chiều và trích xuất bằng chứng vi phạm	Error! Bookmark not defined.
Hình 3.9 Giao diện Trợ lý ảo AI tương tác trả lời thông tin thời tiết và kẹt xe	47
Hình 3.10 Minh họa khả năng duy trì ID của ByteTrack	49
Hình 3.11 Nhận diện biển số (OCR)	51

DANH MỤC BẢNG BIỂU

Bảng 1.1 So sánh đặc điểm của các mô hình Two-stage và One-stage Detector.....	5
Bảng 1.2 Cấu trúc các phân hệ cốt lõi của kiến trúc mạng YOLOv8	7
Bảng 1.3 Các thông số kỹ thuật cấu hình của các phiên bản mạng YOLOv8	9
Bảng 1.4 So sánh thuật toán ByteTrack và các phương pháp Tracking truyền thống ..	12
Bảng 1.5 Các công nghệ nền tảng cấu thành hệ thống	13
Bảng 2.1 Quy trình 5 giai đoạn tuần tự trong luồng xử lý dữ liệu AI Pipeline.....	16
Bảng 2.2 Cấu hình siêu tham số (Hyperparameters) cho quá trình huấn luyện YOLOv8s.....	20
Bảng 2.3 Lược đồ cơ sở dữ liệu bảng traffic_events lưu trữ vi phạm.....	30
Bảng 3.1 Cấu hình môi trường hạ tầng phần cứng và phần mềm thực nghiệm	36

DANH MỤC CÁC TỪ VIẾT TẮT VÀ GIẢI THÍCH CÁC THUẬT NGỮ

Từ viết tắt	Giải nghĩa thuật ngữ
AI	Artificial Intelligence (Trí tuệ nhân tạo)
API	Application Programming Interface (Giao diện lập trình ứng dụng)
CCTV	Closed-Circuit Television (Camera giám sát an ninh)
CNN	Convolutional Neural Network (Mạng nơ-ron tích chập)
FPS	Frames Per Second (Tốc độ khung hình trên giây)
IoU	Intersection over Union (Chỉ số đo lường độ trùng khớp không gian)
ITS	Intelligent Transportation System (Hệ thống giao thông thông minh)
mAP	Mean Average Precision (Độ chính xác trung bình)
MOT	Multi-Object Tracking (Theo dõi đa đối tượng)
OCR	Optical Character Recognition (Nhận dạng ký tự quang học)
RegEx	Regular Expression (Biểu thức chính quy)

ROI	Region of Interest (Vùng không gian quan tâm/Vùng kiểm soát)
RTSP	Real-Time Streaming Protocol (Giao thức truyền tải video thời gian thực)
SAHI	Slicing Aided Hyper Inference (Kỹ thuật băm ảnh hỗ trợ suy luận vật thể nhỏ)
YOLO	You Only Look Once (Thuật toán nhận diện đối tượng theo thời gian thực)

CHƯƠNG 1 TỔNG QUAN ĐỀ TÀI VÀ CƠ SỞ LÝ THUYẾT DEEP LEARNING

1.1 Đặt vấn đề và Mục tiêu nghiên cứu

1.1.1 Đặt vấn đề

Cùng với sự phát triển mạnh mẽ của quá trình đô thị hóa, gia tăng dân số và sự bùng nổ của phương tiện giao thông cá nhân, bài toán quản lý và điều hành giao thông tại các thành phố lớn đang trở thành một thách thức mang tính cấp thiết. Tình trạng ùn tắc giao thông và các hành vi vi phạm luật lệ giao thông (như chạy quá tốc độ, đi ngược chiều, vượt đèn đỏ) không chỉ gây thiệt hại lớn về kinh tế, làm giảm chất lượng cuộc sống mà còn đe dọa trực tiếp đến an toàn tính mạng của người tham gia giao thông.

Hiện nay, tại nhiều nút giao và tuyến đường trọng điểm, hệ thống camera giám sát (CCTV) đã được lắp đặt rộng rãi. Tuy nhiên, phần lớn các hệ thống này mới chỉ dừng lại ở mức độ ghi hình thụ động. Việc trích xuất dữ liệu, phát hiện vi phạm và điều phối giao thông vẫn phụ thuộc rất nhiều vào sức lực của con người trực tiếp quan sát qua màn hình. Phương pháp truyền thống này bộc lộ nhiều hạn chế: chi phí vận hành cao, dễ xảy ra sai sót do yếu tố chủ quan (sự mệt mỏi, mất tập trung của người trực ban), và hoàn toàn không khả thi khi phải giám sát hàng ngàn luồng camera cùng lúc theo thời gian thực (Real-time).

Trong bối cảnh đó, sự trỗi dậy của Trí tuệ nhân tạo (AI), đặc biệt là nhánh Học sâu (Deep Learning) trong lĩnh vực Thị giác máy tính (Computer Vision), đã mở ra hướng giải quyết triệt để cho bài toán này. Các mô hình mạng nơ-ron chập (CNN) hiện đại cho phép máy tính "nhìn" và "hiểu" video tương tự như con người, với tốc độ và độ chính xác vượt trội.

Xuất phát từ thực tiễn trên, đề tài "Xây dựng hệ thống giám sát giao thông thông minh phát hiện, theo dõi và phân tích hành vi vi phạm phương tiện dựa trên nền tảng Deep Learning" được thực hiện nhằm chuyển đổi các camera thụ động thành những "mắt thần" chủ động. Việc nghiên cứu ứng dụng AI vào phân tích hành vi giao thông không chỉ góp phần nâng cao ý thức tự giác của người dân mà còn cung cấp bộ công cụ đắc

lực hỗ trợ các cơ quan chức năng trong việc điều phối và quản lý hệ thống Giao thông thông minh (ITS).



Hình 1.1 Trung tâm giám sát giao thông

(Nguồn: Báo tuổi trẻ online)

1.1.2 Mục tiêu nghiên cứu

Mục tiêu tổng quát: Nghiên cứu, thiết kế và xây dựng một hệ thống phần mềm hoàn chỉnh, có khả năng tích hợp với các camera giao thông hiện có để tự động nhận diện phương tiện, theo dõi quỹ đạo di chuyển và phân tích các hành vi vi phạm luật giao thông theo thời gian thực dựa trên các mô hình Học sâu (Deep Learning).

Mục tiêu cụ thể: Để hoàn thành mục tiêu tổng quát, đề tài tập trung giải quyết các bài toán cụ thể sau:

Về mặt Trí tuệ nhân tạo:

- Nghiên cứu, huấn luyện và tối ưu hóa mô hình nhận diện đối tượng họ YOLO (tiêu biểu là YOLOv8) để nhận diện phân loại chính xác các loại phương tiện (ô tô, xe máy, xe tải, xe buýt) trong nhiều điều kiện môi trường.
- Áp dụng các thuật toán theo dõi đa đối tượng (Multi-Object Tracking - MOT) để

gán và duy trì định danh (ID) duy nhất cho từng phương tiện trên các khung hình.

- Xây dựng thuật toán hình học và logic để phân tích hành vi xe dựa trên Vùng quan tâm (Region of Interest - ROI), từ đó tự động bắt các lỗi: chạy quá tốc độ, đi ngược chiều và vượt đèn đỏ.

Về mặt Kỹ thuật phần mềm:

- Thiết kế kiến trúc hệ thống Client-Server đảm bảo khả năng xử lý luồng Video Streaming (RTSP/HLS/WebRTC) ổn định, chịu tải tốt.
- Xây dựng cơ sở dữ liệu trên nền tảng điện toán đám mây để lưu trữ vĩnh viễn các bản ghi vi phạm và dữ liệu cấu hình tọa độ camera.
- Phát triển giao diện quản trị và trung tâm giám sát trực quan đa nền tảng, cho phép người vận hành thao tác vẽ vùng ROI trực tiếp lên video và xem các biểu đồ thống kê lưu lượng giao thông theo thời gian thực.

1.2 Cơ sở lý thuyết về Deep Learning trong nhận diện hình ảnh

1.2.1 Mạng nơ-ron tích chập

Trong kỷ nguyên của Học sâu (Deep Learning), mạng nơ-ron tích chập (CNN) đã trở thành kiến trúc nền tảng và đóng vai trò quyết định trong sự phát triển của các bài toán Thị giác máy tính [1]. Đối với cấu trúc mạng nơ-ron truyền thống (Fully Connected Network), mỗi pixel của hình ảnh đầu vào đều được kết nối với mọi nơ-ron ở tầng kế tiếp. Cách tiếp cận này làm bùng nổ số lượng tham số (Weights), dẫn đến hiện tượng quá khớp (Overfitting) và tiêu tốn tài nguyên tính toán khổng lồ. [2]

CNN giải quyết triệt để hạn chế trên nhờ vào cơ chế cốt lõi: kết nối cục bộ (Local Receptive Fields) và chia sẻ tham số (Shared Weights). Một kiến trúc CNN tiêu chuẩn được cấu thành từ sự đan xen tuần tự của các tầng:

- Tầng tích chập (Convolutional Layer): Các bộ lọc (Kernels) trượt qua không gian ảnh để tính toán tích vô hướng, giúp trích xuất các đặc trưng từ cấp độ thấp (đường nét, góc cạnh) đến cấp cao (hình dáng phương tiện).
- Hàm kích hoạt (Activation Function): Áp dụng các hàm phi tuyến (như ReLU, SiLU) để phá vỡ tính tuyến tính, cho phép mạng học các hàm phức tạp.
- Tầng gom cụm (Pooling Layer): Giảm chiều không gian của bản đồ đặc trưng, tăng tốc độ tính toán và tạo ra tính bất biến với các phép dịch chuyển.

- Tầng kết nối đầy đủ (Fully Connected Layer): Tổng hợp đặc trưng để xuất ra kết quả phân loại cuối cùng.

1.2.2 Bài toán Phát hiện đối tượng

Phát hiện đối tượng yêu cầu AI không chỉ phân loại phương tiện mà còn phải xác định tọa độ khung bao (Bounding Box) của chúng. Về bản chất toán học, bài toán này đòi hỏi mạng nơ-ron phải thực hiện đồng thời hai hàm mục tiêu và xuất ra một vector dự đoán y cho mỗi đối tượng tiềm năng:

$$y = [p_c, b_x, b_y, b_w, b_h, c_1, c_2, \dots, c_k] \quad (1-1)$$

Trong đó:

- $p_c \in [0, 1]$: Điểm tự tin (Confidence Score) cho biết xác suất tồn tại một đối tượng thực sự bên trong hộp bao.
- (b_x, b_y) : Tọa độ tâm của hộp bao so với lưới không gian ảnh.
- (b_w, b_h) : Kích thước chiều rộng và chiều cao của hộp bao đã được chuẩn hóa.
- $c_1, c_2, \dots, c_k$: Xác suất dự đoán đối tượng đó thuộc về phân lớp k (ví dụ: ô tô, xe máy, xe tải).

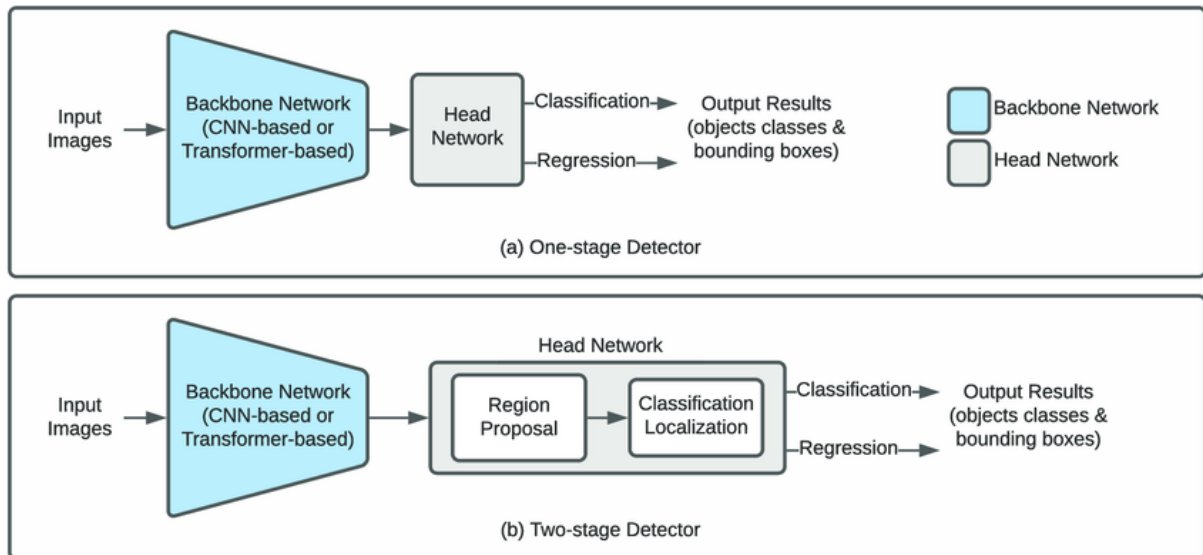
Để đánh giá mức độ chính xác của hộp bao dự đoán (B_{pred}) so với hộp bao thực tế (B_{gt}) do con người gán nhãn, hệ thống sử dụng công thức toán học IoU (Intersection over Union) [3]:

$$IoU = \frac{Area(B_{pred} \cap B_{gt})}{Area(B_{pred} \cup B_{gt})} \quad (1-2)$$

Chỉ số $IoU \in [0, 1]$ càng tiến gần về 1 chứng tỏ mô hình định vị vật thể càng chính xác. Trong Deep Learning, các thuật toán Object Detection giải quyết bài toán tối ưu chỉ số IoU và tính toán vector y này được chia thành hai trường phái tiếp cận chính với các đặc trưng được tổng hợp trong Bảng 1.1.

Bảng 1.1 So sánh đặc điểm của các mô hình Two-stage và One-stage Detector

Tiêu chí so sánh	Two-stage Detectors	One-stage Detectors
Thuật toán tiêu biểu	R-CNN, Fast R-CNN, Faster R-CNN	YOLO (các phiên bản), SSD
Cơ chế hoạt động	Chia làm 2 bước tách biệt: Trích xuất các vùng đề xuất (Region Proposals) → Phân loại và tinh chỉnh khung bao.	Xử lý bài toán như một mô hình hồi quy duy nhất. Dự đoán đồng thời nhãn và khung bao trên toàn bộ ảnh.
Độ chính xác (mAP)	Rất cao, đặc biệt tốt trong việc phát hiện các vật thể có kích thước nhỏ.	Khá cao, các phiên bản YOLO mới (như v8) đã tiệm cận hoặc vượt qua Two-stage.
Tốc độ suy luận (FPS)	Chậm, tiêu tốn nhiều tài nguyên tính toán, khó đạt ngưỡng Real-time (thường < 15 FPS).	Rất nhanh, hỗ trợ tính toán song song, dễ dàng đạt ngưỡng Real-time (30 - 100+ FPS).
Mức độ phù hợp	Phân tích ảnh y tế, ảnh vệ tinh tĩnh.	Giám sát giao thông trực tiếp, xe tự lái.



Hình 1.2 So sánh kiến trúc luồng xử lý của bộ phát hiện đối tượng Two-stage và One-stage.

(Nguồn: Website ResearchGate)

1.3 Kiến trúc mạng YOLOv8

1.3.1 Lịch sử phát triển của họ YOLO

Dòng thuật toán YOLO (You Only Look Once) được khởi xướng bởi Joseph Redmon vào năm 2015, đánh dấu một bước ngoặt vĩ đại cho trường phái One-stage Detector. Trải qua một thập kỷ phát triển liên tục từ các phiên bản từ v1 đến v5, và sau đó được tiếp quản, tối ưu bởi cộng đồng mã nguồn mở và công ty Ultralytics, họ YOLO đã không ngừng tiến hóa để đạt tới đỉnh cao về mặt tốc độ lẫn độ chính xác.

Năm 2023, Ultralytics chính thức phát hành phiên bản YOLOv8. Đây không chỉ đơn thuần là một bản cập nhật cấu trúc, mà là một nền tảng tiên tiến (State-of-the-art - SOTA) được thiết kế lại hoàn toàn. YOLOv8 kế thừa những ưu điểm vượt trội của các phiên bản tiền nhiệm (như khả năng huấn luyện linh hoạt của YOLOv5, cơ chế Anchor-free cải tiến của YOLOvX) đồng thời đưa vào những đột phá mới trong thiết kế phân tách đầu ra, biến nó thành lựa chọn hàng đầu cho các bài toán nhận diện phương tiện giao thông thông minh hiện nay [4].

1.3.2 Cấu trúc cốt lõi của YOLOv8

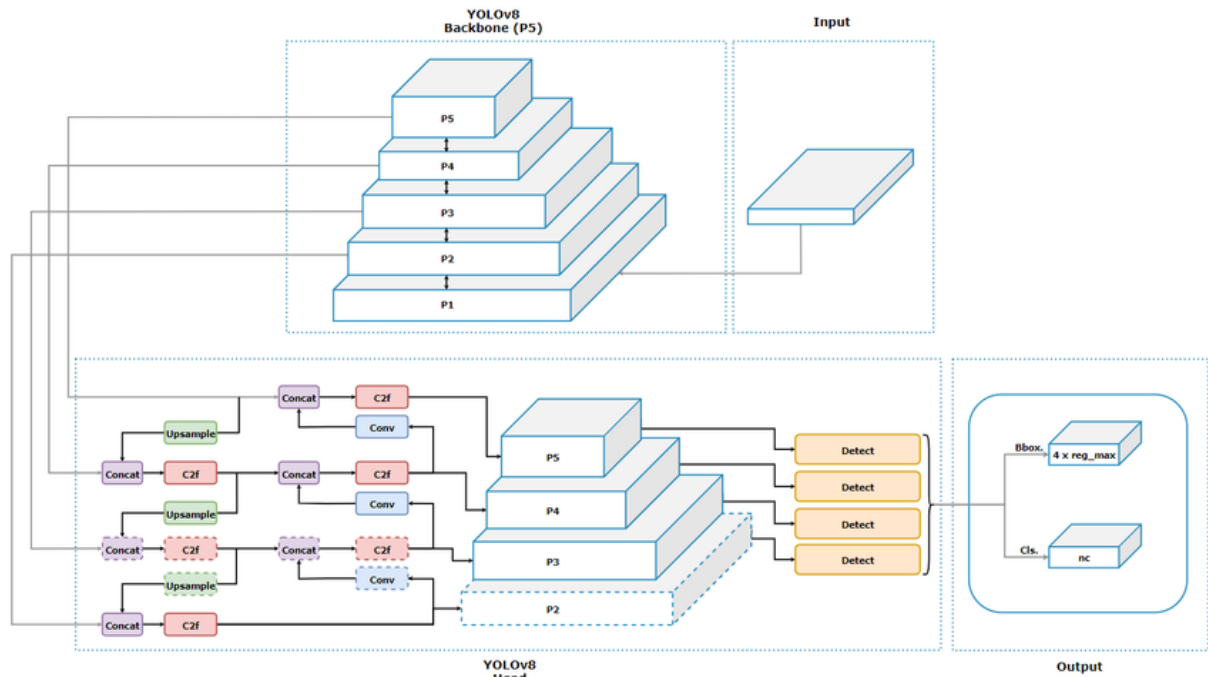
Kiến trúc mạng YOLOv8 được thiết kế lại hoàn toàn so với các thế hệ trước, chia làm 3 phân hệ chức năng rõ rệt: Backbone (Thu thập đặc trưng), Neck (Hòa trộn thông tin) và Head (Dự đoán kết quả).

Để tối ưu hóa việc trình bày, cấu trúc và cải tiến đột phá của từng phân hệ được tóm tắt trong Bảng 1.2 dưới đây:

Bảng 1.2 Cấu trúc các phân hệ cốt lõi của kiến trúc mạng YOLOv8

Phân hệ (Module)	Công nghệ cốt lõi	Vai trò và Hiệu quả mang lại
Backbone (Mạng xương sống)	Mạng CSPDarknet kết hợp khối C2f và SPPF.	Làm nhiệm vụ trích xuất các đặc trưng hình ảnh. Khối C2f giúp lan truyền thông tin hiệu quả hơn mà không làm phình to dung lượng mô hình, kết hợp SPPF để trích xuất nhanh các đặc trưng đa kích thước.
Neck (Mạng cổ)	Kết hợp mạng PANet và FPN.	Hòa trộn dòng thông tin từ các tầng phân giải khác nhau. Việc này cực kỳ hữu ích để nhận diện đồng thời các phương tiện có kích thước chênh lệch lớn trên cùng khung hình (như xe máy nhỏ đi cạnh xe tải lớn).
Head (Mạng đầu ra)	Kiến trúc Anchor-free (Không dùng hộp neo) và Decoupled Head (Đầu tách biệt).	Cải tiến đột phá nhất của v8. Hệ thống tách biệt hoàn toàn luồng dự đoán nhãn (Classification) và luồng dự đoán tọa độ (Regression). Giúp giảm

		hẳn dự đoán sai lệch, tối ưu hàm tổn thất và đẩy nhanh tốc độ hội tụ.
--	--	---



Hình 1.3 Sơ đồ kiến trúc chi tiết của mô hình mạng YOLOv8. [5]

1.3.3 Lý do lựa chọn phiên bản YOLOv8 Small

Trong hệ sinh thái YOLOv8, Ultralytics cung cấp 5 phiên bản mô hình với quy mô tăng dần: Nano (n), Small (s), Medium (m), Large (l), và Extra-Large (x). Việc lựa chọn phiên bản nào để áp dụng vào hệ thống giám sát giao thông thực tế là một bài toán cân bằng khắt khe (Trade-off) giữa Tốc độ xử lý (FPS) và Độ chính xác (mAP).

Ultralytics cung cấp 5 phiên bản YOLOv8 với quy mô mạng nơ-ron tăng dần. Việc lựa chọn phiên bản để tích hợp vào hệ thống giám sát giao thông thực tế là bài toán cân bằng khắt khe (Trade-off) giữa Tốc độ xử lý (FPS) và Độ chính xác (mAP). Bảng 1.3 dưới đây mô tả các thông số kỹ thuật cơ bản để làm căn cứ lựa chọn.

Bảng 1.3 Các thông số kỹ thuật cấu hình của các phiên bản mạng YOLOv8

Phiên bản	Kích thước ảnh đầu vào	Số lượng tham số (Parameters)	Số phép toán tính toán (FLOPs)	Đánh giá tính thực tiễn
YOLOv8n (Nano)	640 X 640	≈ 3.2 Triệu	≈ 8. Tỷ	Tốc độ cực nhanh nhưng độ chính xác thấp, dễ mất dấu xe khi mật độ giao thông đông.
YOLOv8s (Small)	640 X 640	≈ 11.2 Triệu	≈ 28.6 Tỷ	Cân bằng hoàn hảo. Đủ nhẹ để chạy Real-time nhưng đủ sâu để học đặc trưng xe cộ Việt Nam.
YOLOv8m (Medium)	640 X 640	≈ 25.9 Triệu	≈ 78.9 Tỷ	Cần GPU rời cấu hình khá để chạy mượt mà.
YOLOv8l (Large)	640 X 640	≈ 43.7 Triệu	≈ 165.2 Tỷ	Quá nặng, không phù hợp cho suy luận luồng Video trực tiếp trên máy tầm trung.

Dựa vào Bảng 1.2, phiên bản YOLOv8 Small (YOLOv8s) được chọn làm nhân cho toàn bộ hệ thống vì nó đảm bảo tốc độ suy luận ổn định nhưng vẫn cung cấp chất lượng đầu ra bounding box đủ sắc nét để cung cấp cho thuật toán Tracking ở giai đoạn sau.

Độ chính xác của mô hình được đo lường bằng công thức mAP (Mean Average Precision) trên tất cả các lớp đối tượng N:

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^N \text{AP}_i \quad (1-3)$$

Trong khi đó, để hệ thống đạt chuẩn giám sát thời gian thực (Real-time Tracking), tốc độ khung hình bắt buộc phải đạt $\text{FPS} \geq 30$. Suy ra, tổng thời gian xử lý một khung hình (T_{total}) bị giới hạn ngặt nghèo bởi công thức:

$$T_{\text{total}} = T_{\text{preprocess}} + T_{\text{inference}} + T_{\text{postprocess}} \leq \frac{1000 \text{ ms}}{30} \approx 33.3 \text{ ms} \quad (1-4)$$

Với khối lượng tính toán 28.6 GFLOPs, YOLOv8s cung cấp khả năng biểu diễn đặc trưng sâu sắc, phân loại chính xác các dòng phương tiện hỗn hợp tại Việt Nam, đồng thời vẫn giữ được tốc độ suy luận dưới 20 ms/frame, tạo tiền đề hoàn hảo để cấp dữ liệu tọa độ cho giải thuật ByteTrack hoạt động ở các bước tiếp theo.

1.4 Bài toán Theo dõi đa đối tượng

1.4.1 Khái niệm và những thách thức trong bài toán theo dõi đa đối tượng

Nếu như bài toán Phát hiện đối tượng (Object Detection) chỉ chịu trách nhiệm xác định vị trí và phân loại phương tiện tại từng khung hình đơn lẻ (Static Frame), thì bài toán Theo dõi đa đối tượng (Multi-Object Tracking - MOT) lại đóng vai trò duy trì sự liên tục của các đối tượng đó theo thời gian qua một chuỗi các khung hình. Mục tiêu của MOT là gán một định danh duy nhất (Unique ID) cho từng phương tiện xuất hiện trong video và theo dõi sát sao quỹ đạo di chuyển (Trajectory) của chúng cho đến khi chúng rời khỏi góc quét của camera.

Trong môi trường giao thông thực tế tại Việt Nam, thuật toán MOT phải đối mặt với những thách thức vô cùng khắc nghiệt:

- Hiện tượng che khuất (Occlusion): Các phương tiện di chuyển với mật độ dày đặc, xe lớn (xe buýt, xe tải) che khuất hoàn toàn hoặc một phần xe nhỏ (xe máy, xe đạp) khiến mô hình AI bị mất dấu đối tượng tạm thời.
- Sự thay đổi về ngoại hình và ánh sáng: Phương tiện thay đổi hình dáng khi thay đổi góc nhìn đối với camera, hoặc sự thay đổi đột ngột của điều kiện ánh sáng (bóng râm, ánh đèn pha ban đêm, thời tiết mưa gió).
- Hiện tượng ID Switch (Tráo đổi ID): Khi hai phương tiện di chuyển quá sát nhau hoặc đan chéo qua nhau, thuật toán rất dễ bị nhầm lẫn và hoán đổi ID của hai phương tiện này cho nhau, dẫn đến sai lệch nghiêm trọng cho các logic phân tích hành vi vi phạm phía sau.

1.4.2 Giải thuật Tracking ByteTrack

Để giải quyết các thách thức trên, hệ thống ứng dụng giải thuật ByteTrack – thuật toán theo dõi đa đối tượng tiên tiến dựa trên cơ chế liên kết dữ liệu hai giai đoạn. Hầu hết các phương pháp tracking truyền thống chỉ giữ lại các hộp bao (bounding box) có độ tự tin cao (>0.5). Điều này dẫn đến nhược điểm chí mạng: khi phương tiện bị che khuất một phần, điểm tự tin của YOLOv8 trả về sẽ giảm xuống (ví dụ: 0.3), thuật toán cũ sẽ xóa bỏ hộp bao này, gây ra hiện tượng đứt gãy quỹ đạo và tráo đổi ID. ByteTrack khắc phục bằng cơ chế liên kết BYTE [6], sử dụng thuật toán Hungary để tối ưu hóa ma trận chi phí (Cost Matrix) dựa trên chỉ số giao nhau IoU. Bài toán phân công tối ưu giữa tập hợp các quỹ đạo (Tracks) và tập hợp hộp bao phát hiện (Detections) được giải quyết thông qua việc cực tiểu hóa hàm chi phí C :

$$C = 1 - \text{IoU}(B_{\text{track}}, B_{\text{detect}}) \quad (1-5)$$

Quá trình này diễn ra qua hai giai đoạn:

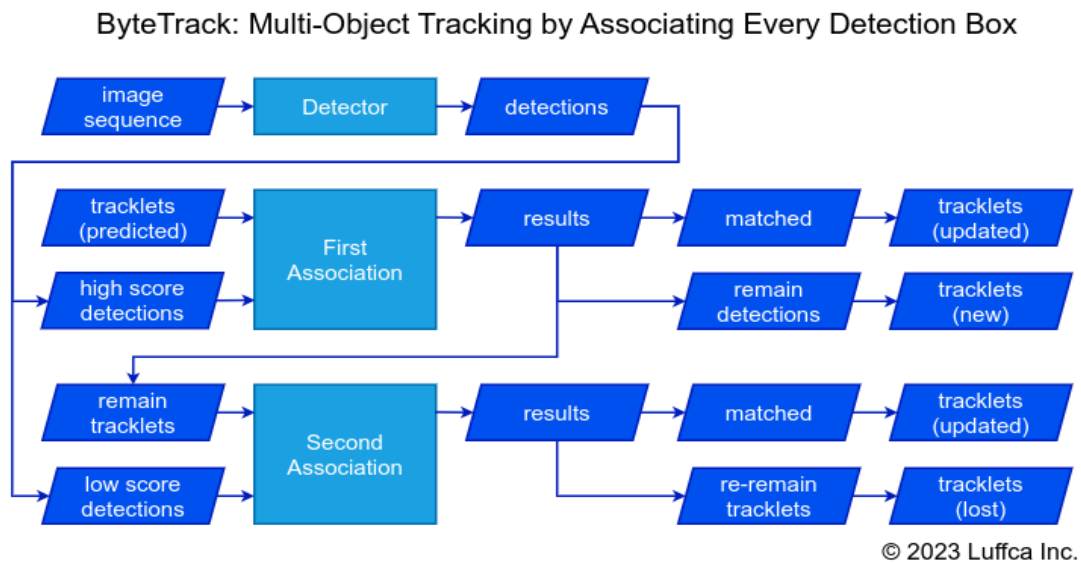
- Giai đoạn 1 (Liên kết hộp tự tin cao): Thuật toán tiến hành so khớp các bounding box có độ tự tin cao với các quỹ đạo hiện có.
- Giai đoạn 2 (Liên kết hộp tự tin thấp): Đối với các quỹ đạo chưa tìm được hộp bao khớp ở giai đoạn 1, ByteTrack không xóa chúng đi mà tiếp tục lấy các bounding box có độ tự tin thấp (từ 0.1 đến 0.5) để so khớp lại. Nếu phát hiện ra sự trùng

khớp không gian, hệ thống hiểu rằng phương tiện chỉ đang bị che khuất tạm thời và lập tức duy trì ID cũ.

Sự khác biệt vượt trội của ByteTrack so với các thuật toán tiền nhiệm được thể hiện rõ qua Bảng 1.4 dưới đây:

Bảng 1.4 So sánh thuật toán ByteTrack và các phương pháp Tracking truyền thống

Tiêu chí	SORT	DeepSORT	ByteTrack
Tiêu chí liên kết	Chỉ dùng IoU và Kalman Filter.	Thêm đặc trưng ngoại hình (Re-ID).	IoU linh hoạt kết hợp phân vùng điểm số tự tin.
Xử lý hộp bao điểm thấp	Xóa bỏ ngay lập tức.	Xóa bỏ ngay lập tức.	Giữ lại và đưa vào so khớp ở giai đoạn 2.
Hiệu năng chống che khuất	Rất kém, dễ đứt quỹ đạo.	Khá, nhưng tốn tài nguyên tính Re-ID.	Xuất sắc, nối lại quỹ đạo ngay cả khi bị che lấp nặng.
Tốc độ (FPS)	Rất nhanh.	Chậm (do trích xuất đặc trưng hình ảnh).	Rất nhanh (Chỉ dùng toán học hình học 2D).



Hình 1.4 Quy trình liên kết dữ liệu hai giai đoạn của giải thuật ByteTrack.

(Nguồn: Luffca Inc , 2023)

1.5 Cơ sở lý thuyết về Nhận dạng ký tự quang học

Trong hệ thống giao thông thông minh, nếu Object Detection và Tracking đóng vai trò định vị phương tiện, thì OCR (Optical Character Recognition) chính là cốt lõi để định danh chủ phương tiện vi phạm. Bài toán nhận diện biển số xe (ALPR - Automatic License Plate Recognition) thường được chia làm hai giai đoạn toán học khép kín:

- **Cắt vùng biển số (License Plate Cropping):** Dựa trên tọa độ bounding box của mô hình Detection, hệ thống tiến hành cắt (Crop) và áp dụng các phép biến đổi hình thái học (Morphological transformations) như nhị phân hóa (Binarization) để làm nổi bật độ tương phản giữa các ký tự (chữ đen) và nền biển số (nền trắng/vàng).
- **Trích xuất chuỗi ký tự (Character Recognition):** Vùng ảnh chứa biển số được nạp vào mạng nơ-ron học sâu chuyên biệt (thường là sự kết hợp giữa CNN để trích xuất đặc trưng hình ảnh và RNN/LSTM để nhận dạng tính tuần tự của chuỗi ký tự). Đầu ra của mô hình là một chuỗi văn bản dạng String (ví dụ: 51F-123.45) được lưu trực tiếp vào cơ sở dữ liệu làm bằng chứng phạt nguội.

1.6 Các nền tảng hỗ trợ đóng gói hệ thống

Để hiện thực hóa các giải thuật toán học phức tạp thành một sản phẩm có tính ứng dụng cao, kiến trúc phần mềm được thiết kế theo mô hình Client-Server hiện đại. Đặc biệt, để đảm bảo năng lực phản hồi thời gian thực, toàn bộ mô hình Deep Learning cốt lõi đều được thiết kế tích hợp và tải sẵn vào bộ nhớ ngay tại thời điểm khởi động hệ thống.

Bảng 1.5 Các công nghệ nền tảng cấu thành hệ thống

Lớp kiến trúc	Công nghệ sử dụng	Vai trò và Cách thức hoạt động
Backend API & AI Core	Python (FastAPI)	Đóng vai trò máy chủ xử lý trung tâm. Khai thác cơ chế bất đồng bộ (async/await) để xử lý luồng streaming [7]. Mô hình AI được tích hợp trọn

		vện và nạp vào RAM ngay lúc đầu khởi động máy chủ, triệt tiêu hoàn toàn độ trễ (cold-start) khi có video truyền tới.
Frontend Dashboard	ReactJS & Vite	Xây dựng giao diện người dùng (UI/UX) với khả năng render biểu đồ tức thời [8]. Tích hợp trực tiếp nút tải ảnh và video tĩnh trên giao diện, cho phép người dùng linh hoạt tải dữ liệu lên để hệ thống phân tích ngoại tuyến thay vì chỉ phụ thuộc vào camera trực tiếp.
Database	PostgreSQL (Supabase)	Hệ quản trị cơ sở dữ liệu quan hệ trên đám mây, lưu trữ vĩnh viễn tọa độ camera, log vi phạm và kết quả biến số xe từ OCR với tính toàn vẹn dữ liệu cao.
Hệ sinh thái API bên thứ ba	TomTom Traffic API, Telegram Bot API, Google Gemini API & WeatherAPI..	TomTom API: Trích xuất dữ liệu vận tốc dòng xe vĩ mô theo thời gian thực để phân tích mức độ ùn tắc và vẽ bản đồ nhiệt (Heatmap).

		Telegram Bot API: Hoạt động như một cổng thông báo bất đồng bộ, lập tức đẩy tin nhắn văn bản và hình ảnh phương tiện vi phạm đến thiết bị di động của lực lượng điều hành. Gemini & WeatherAPI: Tích hợp mô hình ngôn ngữ lớn (LLM) và dữ liệu khí tượng để cấp "bộ não" cho Trợ lý ảo (Chatbot), hỗ trợ truy vấn thông tin đa chiều bằng ngôn ngữ tự nhiên.
--	--	--

CHƯƠNG 2 :XÂY DỰNG, HUẤN LUYỆN MÔ HÌNH VÀ THIẾT KẾ HỆ THỐNG

2.1 Kiến trúc luồng xử lý dữ liệu AI

2.1.1 Luồng tuần tự của hệ thống nhận diện và phân tích hành vi

Để hệ thống có thể phân tích và đưa ra các quyết định xử lý đối với những hành vi vi phạm phức tạp như chạy quá tốc độ, vượt đèn đỏ hay đi ngược chiều từ một luồng video thô, dữ liệu bắt buộc phải được truyền dẫn và xử lý tuần tự qua một đường ống công nghệ khép kín (AI Pipeline).

Mỗi khung hình của video khi đi qua đường ống này sẽ lần lượt trải qua 5 giai đoạn xử lý cốt lõi, trong đó đầu ra của giai đoạn trước chính là điều kiện đầu vào bắt buộc cho giai đoạn sau. Quy trình tuần tự hóa dữ liệu được mô tả chi tiết trong Bảng 2.1 dưới đây:

Bảng 2.1 Quy trình 5 giai đoạn tuần tự trong luồng xử lý dữ liệu AI Pipeline

Thứ tự	Giai đoạn xử lý	Công nghệ & Giải thuật áp dụng	Chức năng và Dữ liệu Đầu ra (Output)
1	Thu nhận dữ liệu (Data Ingestion)	Giao thức RTSP, HLS, thư viện giải mã OpenCV.	Tiếp nhận luồng video thô. Bẻ gãy dòng video thành các ma trận điểm ảnh (khung hình đơn lẻ) theo tần suất khung hình (FPS) của nguồn phát.
2	Phát hiện phương tiện (Vehicle Detection)	Mạng học sâu YOLOv8 Small.	Quét khung hình, phân loại đối tượng. Đầu ra: Tọa độ khung bao cục bộ (x_{min} , y_{min} , x_{max} , y_{max}), độ tự tin (Confidence Score) và nhãn lớp (Car, Bike, Bus, Truck).

3	Theo dõi đa đối tượng (Multi-Object Tracking)	Giải thuật ByteTrack, Thuật toán Hungary, Chỉ số IoU.	Đối sánh vật thể hiện tại với quỹ đạo ở các khung hình trước. Đầu ra: Gán cố định một chỉ số ID duy nhất (VD: Car ID 12) xuyên suốt hành trình của phương tiện.
4	Trích xuất biển số (OCR Engine)	YOLOv8-License-Plate, Tesseract/EasyOCR, Biểu thức chính quy (RegEx).	Trích xuất ảnh biển số từ Bounding Box. Đọc chữ và dùng RegEx chuẩn hóa format biển số Việt Nam. Đầu ra: Chuỗi ký tự biển số (VD: 51F12345).
5	Phân tích hành vi & Lưu trữ (Behavior & Storage)	Giải thuật Point-in-Polygon, Vector toán học, FastAPI, PostgreSQL.	Đối chiếu quỹ đạo ID với tọa độ vùng ROI và quy tắc giao thông. Đầu ra: Kích hoạt lệnh chụp ảnh vi phạm và đẩy bản ghi trực tiếp xuống Cloud Database.

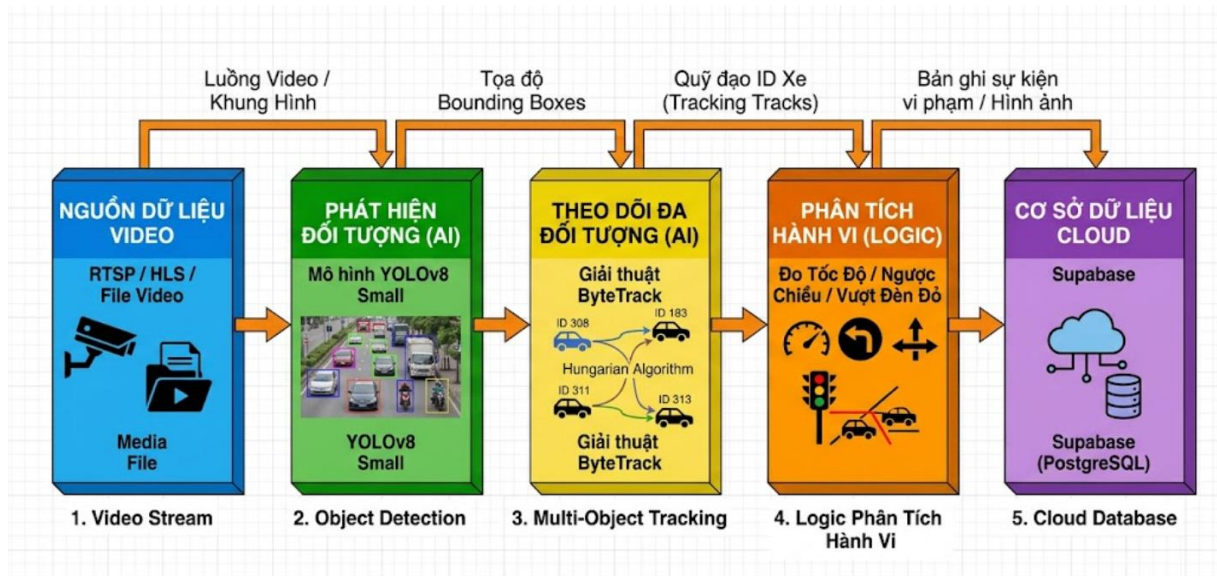
2.1.2 Sơ đồ khối tổng thể của hệ thống

Mối quan hệ tương tác và luồng di chuyển của dữ liệu giữa các thành phần công nghệ trong Pipeline AI được biểu diễn trực quan thông qua sơ đồ kiến trúc hệ thống dưới đây:

Video Stream / RTSP → YOLOv8 Detector → ByteTrack Tracker → Behavior Analysis Logic → PostgreSQL Database

Kiến trúc luồng xử lý này đảm bảo tính toàn vẹn của dữ liệu và tối ưu hóa tối đa hiệu năng tính toán. Việc tách biệt các module chức năng (Phát hiện → Theo dõi → Phân tích) giúp hệ thống dễ dàng nâng cấp, bảo trì, đồng thời tận dụng được tối đa sức mạnh

xử lý bất đồng bộ (Asynchronous) của framework FastAPI ở Backend, giúp giảm thiểu độ trễ tối đa khi xử lý các luồng dữ liệu giao thông trực tiếp.



Hình 2.1 Sơ đồ khối tổng thể luồng kiến trúc pipeline xử lý dữ liệu AI của hệ thống.

(Nguồn: Tác giả tự thiết kế qua Gemini AI)

2.2 Huấn luyện và Tối ưu mô hình YOLOv8

2.2.1 Chuẩn bị và tiền xử lý tập dữ liệu

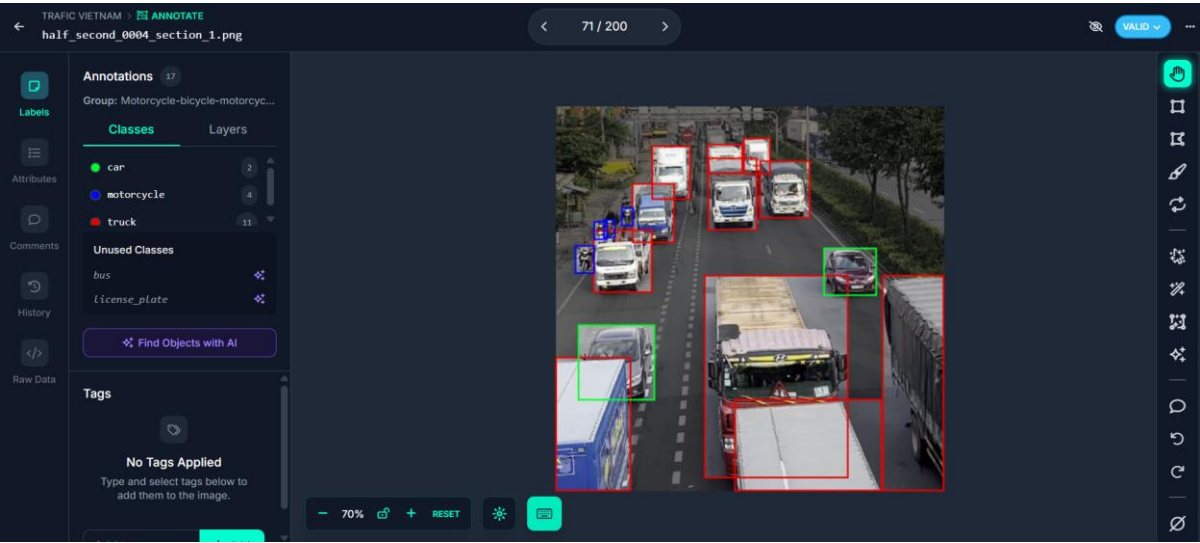
Để mô hình YOLOv8 Small có khả năng nhận diện chính xác các loại phương tiện trong môi trường giao thông đặc thù tại Việt Nam (nơi có mật độ xe máy chiếm tỷ trọng áp đảo và dòng giao thông hỗn hợp), việc chuẩn bị một tập dữ liệu (Dataset) huấn luyện đa dạng là yếu tố quyết định. Tập dữ liệu sử dụng trong đề tài được thu thập từ các đoạn video thực tế trích xuất tại các nút giao trọng điểm ở TP.HCM và được định dạng theo chuẩn YOLO: <class_id> <x_center> <y_center> <width> <height>.

Trong đó, tọa độ được chuẩn hóa (Normalize) về dải giá trị [0, 1] so với kích thước gốc của bức ảnh theo công thức:

$$x_{center} = \frac{x_{absolute}}{Image_Width}, \quad y_{center} = \frac{y_{absolute}}{Image_Height} \quad (2-1)$$

Hệ thống tập trung phân loại 5 lớp đối tượng (Classes) chính, bao gồm: car (13,080 nhãn), motorcycle (26,657 nhãn), truck (5,465 nhãn), bus (1,021 nhãn) và

license_plate (1,053 nhãn biển số). Việc gán nhãn được thực hiện tỉ mỉ thông qua nền tảng Roboflow.



Hình 2.2 Minh họa quá trình gán nhãn tọa độ khung bao cho các phương tiện giao thông.

(Nguồn: Bối tác giả)

☰ Classes & Tags

Classes 5 Tags 1 [What is a class?](#)

Search classes... ☐ Lock Classes Sort By Class Ascending ▾

COLOR	CLASS NAME	COUNT	MODIFY
●	bus	1,021	☐
●	car	13,080	☐
●	license_plate	1,053	☐
●	motorcycle	26,657	☐
●	truck	5,465	☐

Hình 2.3 Phân bố số lượng nhãn các lớp đối tượng trên nền tảng Roboflow.

(Nguồn: Bối tác giả)

2.2.2 Kỹ thuật tăng cường dữ liệu và cấu hình siêu tham số

Môi trường giao thông thực tế thường xuyên biến động do yếu tố thời tiết, ánh sáng và rung lắc camera. Để nâng cao năng lực tổng quát hóa của mạng nơ-ron và tránh hiện tượng học vẹt (Overfitting), các kỹ thuật tăng cường dữ liệu tiên tiến tích hợp trong YOLOv8 đã được thiết lập ở mức độ ép xung cao:

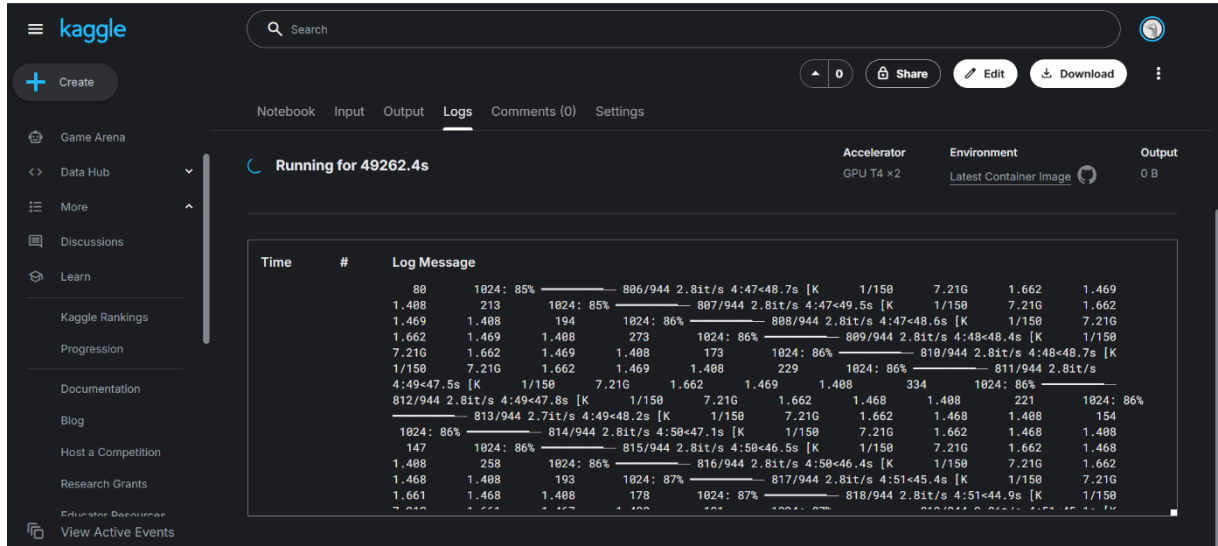
- Mosaic Augmentation (1.0) & Mixup (0.1): Tổ hợp ngẫu nhiên 4 bức ảnh và trộn đè các bức ảnh lên nhau. Kỹ thuật này ép mô hình học cách "nhìn xuyên thấu" và nhận diện phương tiện ngay cả khi bị che khuất nặng nề.
- Spatial Transforms: Áp dụng phép xoay ảnh ngẫu nhiên (10°) để giả lập điều kiện camera bị rung lắc trong thực tế.

Quá trình huấn luyện được thực thi trên môi trường điện toán đám mây Kaggle với phần cứng GPU tăng tốc NVIDIA Tesla T4 (15 GB VRAM). Các siêu tham số (Hyperparameters) được tinh chỉnh chuyên sâu để tối ưu hóa thuật toán, chi tiết tại Bảng 2.2.

Bảng 2.2 Cấu hình siêu tham số (Hyperparameters) cho quá trình huấn luyện YOLOv8s

Nhóm tham số	Tên tham số	Giá trị	Ý nghĩa và Tác dụng thực tiễn
Phần cứng	imgsz	1024	Độ phân giải cao 1024 X 1024 giúp giữ lại đặc trưng của các vật thể rất nhỏ (như biển số xe hay xe máy ở xa).
	batch	8	Kích thước lô dữ liệu phù hợp với dung lượng VRAM của Tesla T4.
Vòng lặp	epochs	150	Số chu kỳ học đủ dài để mô hình hội tụ điểm cực tiểu toàn cục.
	patience	50	Nới lỏng số vòng chờ (Early Stopping) để mô hình có cơ hội bứt phá khỏi các cực tiểu địa phương.
Tối ưu hóa	optimizer	AdamW	Thuật toán tối ưu hóa tiên tiến kết hợp phân rã trọng số (weight_decay=0.0005), giúp mô hình học mượt mà hơn.

	cos_lr	True	Kích hoạt bộ lập lịch tốc độ học theo hàm Cosine, giúp chỉ số mAP50-95 tăng mạnh ở các epoch cuối.
--	--------	------	--



Hình 2.4 Môi trường cấu hình và tiến trình huấn luyện mô hình YOLOv8s trên Kaggle.

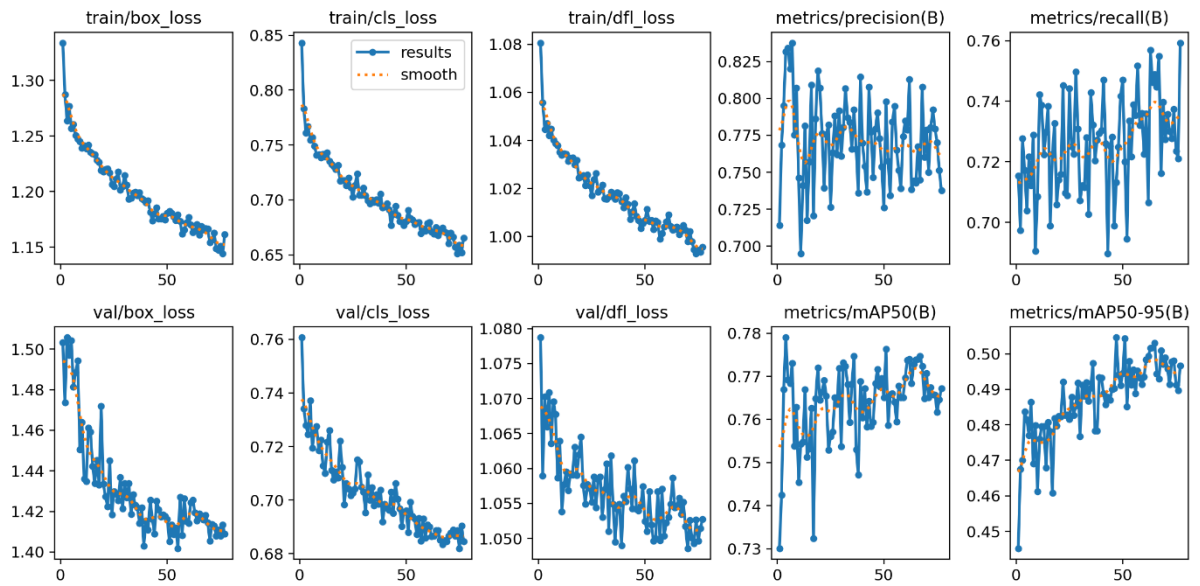
(Nguồn: Bồi tác giả)

2.2.3 Đánh giá kết quả huấn luyện

Sau 150 chu kỳ huấn luyện, mô hình được kiểm thử trên tập Validation độc lập gồm 265 hình ảnh chứa 4,114 thực thể phương tiện. Chỉ số mAP (Mean Average Precision) được sử dụng làm thước đo định lượng cốt lõi:

$$\text{mAP} = \frac{1}{N} \sum_{i=1}^N \text{AP}_i \quad (2-2)$$

Kết quả thực nghiệm trên tập Val cho thấy độ chính xác trung bình mAP50 đạt mức xuất sắc 0.886 (88.6%). Phân tích sâu vào từng lớp: lớp biển số xe (license_plate) đạt mAP50 cao nhất (0.974), theo sau là xe tải (truck - 0.911) và ô tô con (car - 0.907). Tốc độ suy luận (Inference time) đạt 20.9 ms/ảnh, hoàn toàn đáp ứng tiêu chuẩn thời gian thực (Real-time).

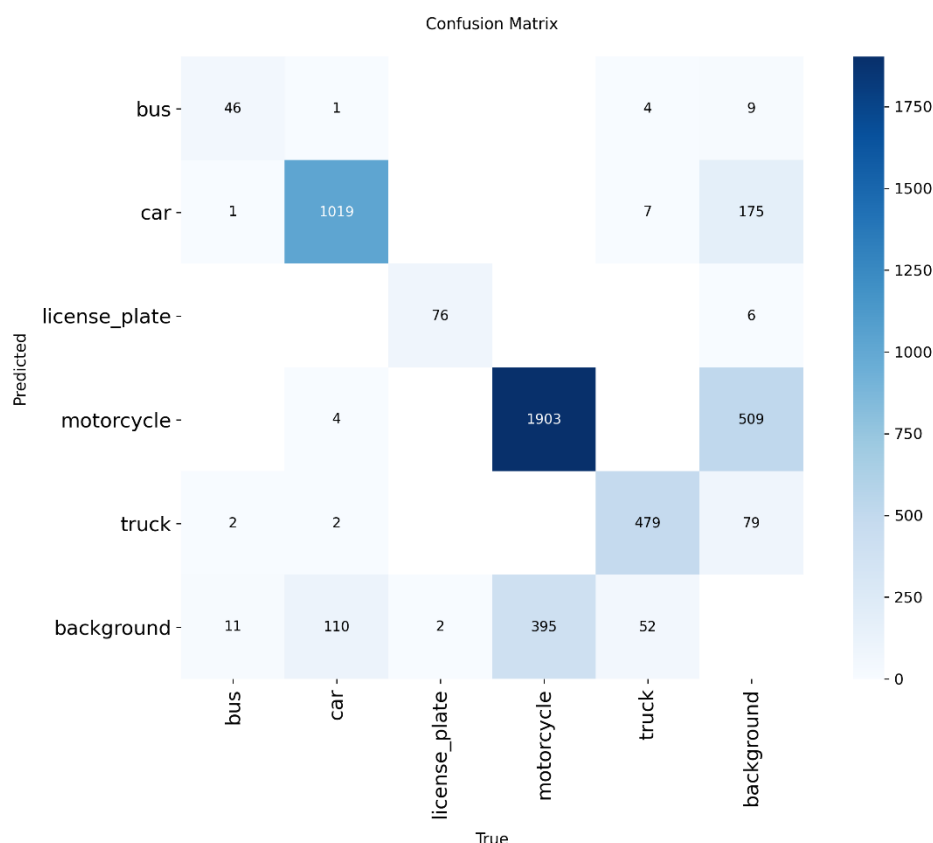


Hình 2.5 Các đường cong hàm mất mát (Loss) và biểu đồ chỉ số mAP đánh giá chất lượng mô hình sau huấn luyện.

(Nguồn: Bời tác giả)

Đặc biệt, để khắc phục triệt để điểm yếu cố hữu của YOLO là bỏ sót các vật thể kích thước siêu nhỏ trong đám đông, hệ thống đề xuất tích hợp thêm kỹ thuật SAHI (Slicing Aided Hyper Inference) trong quá trình suy luận. SAHI hoạt động theo nguyên lý chia nhỏ bức ảnh gốc thành các vùng lưới đè lên nhau (Overlap) trước khi đưa qua mô hình YOLO, sau đó hợp nhất kết quả lại (NMS - Non-Maximum Suppression).

Quá trình A/B Testing thực tế trên ảnh độ phân giải cao chứng minh sự hiệu quả vượt trội: mô hình YOLO tiêu chuẩn chỉ bắt được 30 đối tượng, trong khi YOLO kết hợp thuật toán phân mảnh lưới SAHI (slice_height=512, overlap_ratio=0.2) phát hiện thành công 33 đối tượng, gia tăng độ chính xác trong các tình huống xe che khuất mà không cần thay đổi trọng số mô hình.



Hình 2.6 Ma trận nhầm lẫn (Confusion Matrix) và các biểu đồ phân tích độ chính xác trên từng lớp đối tượng.

(Nguồn: Bồi tác giả)

2.3 Giải thuật hình học phân tích vùng quan tâm

2.3.1 Bản chất toán học của việc thiết lập vùng đa giác kiểm soát

Trong hệ thống Giám sát Giao thông Thông minh (ITS), việc nhận diện và gán ID phương tiện đơn thuần trên màn hình chưa đủ cấu thành điều kiện để bắt lỗi vi phạm. Hệ thống cần phải có một cơ chế xác định xem phương tiện đó có đang thực hiện hành vi xâm phạm vào các phân khu giao thông cấm hay không (ví dụ: khu vực ngã tư đang đèn đỏ, làn đường ngược chiều, hoặc đoạn đường đo tốc độ). Phân khu không gian này được định nghĩa là Vùng quan tâm (ROI - Region of Interest).

Trên giao diện người dùng (ReactJS), người vận hành sử dụng thao tác click chuột để đánh dấu 4 điểm ngẫu nhiên trên màn hình Canvas, tạo thành một đa giác lõm 4 cạnh khép kín. Về mặt toán học, vùng ROI này được biểu diễn dưới dạng một tập hợp mảng (Array) chứa tọa độ chuẩn hóa của 4 đỉnh theo thứ tự nối tiếp:

$$ROI = \{P_1(x_1, y_1), P_2(x_2, y_2), P_3(x_3, y_3), P_4(x_4, y_4)\} \quad (2-3)$$

Trong đó, giá trị $x_i, y_i \in [0, 1]$ tương ứng với tỷ lệ vị trí điểm so với chiều rộng và chiều cao tổng thể của khung hình video. Việc chuẩn hóa này đảm bảo khi luồng stream thay đổi độ phân giải hiển thị trên các kích thước màn hình khác nhau, vùng hình học ROI vẫn không bị méo lệch hay sai vị trí.

2.3.2 Thuật toán kiểm tra điểm nằm trong đa giác

Để xác định một phương tiện đã đi vào vùng cấm hay chưa, tại mỗi khung hình, Backend Python sẽ trích xuất tọa độ điểm đáy (Bottom-center) của khung bao phương tiện – ký hiệu là $M(x_m, y_m)$, đại diện cho điểm tiếp xúc của bánh xe với mặt đường. Hệ thống áp dụng thuật toán hình học máy tính Ray Casting (Tia chiếu) hoặc phương pháp tính Số chỉ quấn (Winding Number) để kiểm tra trạng thái của điểm M đối với đa giác ROI.

Theo nguyên lý toán học của giải thuật Ray Casting, hệ thống giả lập một tia khởi nguồn từ điểm $M(x_m, y_m)$ và kéo dài vô hạn theo phương ngang sang phía bên phải. Hệ thống tiến hành duyệt qua tất cả các cạnh tạo bởi các đỉnh liên tiếp $P_i P_{i+1}$ của đa giác ROI và đếm tổng số lần mà tia ngang này cắt (Intersect) qua các cạnh của đa giác:

- Nếu tổng số giao điểm đếm được là một Số lẻ (1, 3...) $\rightarrow$ Điểm M chắc chắn nằm bên trong đa giác ROI (Phương tiện đang nằm trong vùng kiểm soát).
- Nếu tổng số giao điểm đếm được là một Số chẵn (0, 2...) $\rightarrow$ Điểm M nằm bên ngoài đa giác ROI (Phương tiện nằm ngoài vùng kiểm soát).

Thuật toán Ray Casting có độ phức tạp thời gian chỉ là $O(N)$ với N là số đỉnh của đa giác (ở đây $N=4$). Nhờ sự tối ưu này, Backend có thể kiểm tra trạng thái ROI của hàng trăm phương tiện đồng thời trên mỗi khung hình mà không gây ra bất kỳ độ trễ (latency) nào, duy trì tốc độ khung hình chuẩn Real-time.

Logic cốt lõi của giải thuật Ray Casting được hệ thống thực thi thông qua đoạn mã sau:

```

def is_point_in_polygon(x, y, polygon):
    num_vertices = len(polygon)
    inside = False
    p1x, p1y = polygon[0]

    for i in range(num_vertices + 1):
        p2x, p2y = polygon[i % num_vertices]
        if y > min(p1y, p2y):
            if y <= max(p1y, p2y):
                if x <= max(p1x, p2x):
                    # Tính toán phương trình đường thẳng để tìm giao điểm X
                    if p1y != p2y:
                        x_intersect = (y - p1y) * (p2x - p1x) / (p2y - p1y) + p1x
                    # Nếu tia chiếu cắt qua cạnh, đảo ngược trạng thái inside
                    if p1x == p2x or x <= x_intersect:
                        inside = not inside
        p1x, p1y = p2x, p2y
    return inside

```

Hình 2.7 Giải thuật Ray Casting

(Nguồn: Bời tác giả)

2.4 Đề xuất Logic và Giải thuật phát hiện vi phạm

2.4.1 Giải thuật đo tốc độ phương tiện dựa trên phép biến đổi phối cảnh

Bản chất của hình ảnh thu được từ Camera giám sát là không gian 2D chiếu từ không gian 3D thực tế, dẫn đến hiện tượng biến dạng phối cảnh: hai vật thể có cùng kích thước thực tế nhưng vật ở xa camera sẽ hiển thị nhỏ hơn vật ở gần, và khoảng cách pixel giữa chúng cũng bị thu hẹp lại. Do đó, nếu tính vận tốc bằng cách đếm số pixel dịch chuyển trực tiếp trên màn hình sẽ gây ra sai số cực kỳ lớn.

Để giải quyết bài toán này, hệ thống ứng dụng thuật toán Biến đổi phối cảnh (Perspective Transformation) nhằm chuyển đổi ma trận tọa độ pixel trên camera về ma trận tọa độ phẳng thực tế (hệ mét). Người vận hành sẽ khai báo cấu hình kích thước thực tế của đoạn đường phân tích thông qua hai hằng số: $REAL_WIDTH_M = 14.0$ (Chiều rộng thực tế góc đường, mét) và $REAL_HEIGHT_M = 6.0$ (Chiều dài đoạn đường đo, mét).

Hệ thống thiết lập một ma trận biến đổi M kích thước 3×3 dựa trên 4 điểm phối cảnh trên ảnh và 4 điểm tương ứng trong không gian thực tế thông qua hàm `cv2.getPerspectiveTransform()`. Công thức toán học của phép biến đổi một điểm tọa độ từ camera $A(x_{cam}, y_{cam})$ sang tọa độ thực tế $A'(x_{real}, y_{real})$ được biểu diễn như sau [9]:

$$\begin{bmatrix} x' \\ y' \\ w' \end{bmatrix} = M \times \begin{bmatrix} x_{cam} \\ y_{cam} \\ 1 \end{bmatrix} \quad (2-4)$$

$$x_{real} = \frac{x'}{w'}, \quad y_{real} = \frac{y'}{w'} \quad (2-5)$$

Khi bộ lọc ByteTrack duy trì được ID của xe qua các khung hình, Backend sẽ tính toán khoảng cách Euclidean giữa hai vị trí thực tế của xe tại khung hình hiện tại t và khung hình trước đó $t-1$:

$$d = \sqrt{(x_{real_t} - x_{real_t-1})^2 + (y_{real_t} - y_{real_t-1})^2} \quad (2-6)$$

Vận tốc tức thời của phương tiện (km/h) được xác định bằng công thức:

$$v = \left(\frac{d}{\Delta t} \right) \times 3.6 = (d \times FPS) \times 3.6 \quad (2-7)$$

Trong đó $\Delta t = \frac{1}{FPS}$ là khoảng thời gian giãn cách giữa hai khung hình liên tiếp. Nếu vận tốc v vượt quá ngưỡng cấu hình speedLimit (ví dụ: 55km/h), hệ thống lập tức gắn cờ vi phạm cho ID phương tiện đó.

2.4.2 Giải thuật phát hiện phương tiện đi ngược chiều

Bài toán phát hiện đi ngược chiều được giải quyết bằng phương pháp Phân tích Vector hướng di chuyển của phương tiện dựa trên lịch sử vết quỹ đạo mà ByteTrack cung cấp.

Khi một phương tiện xuất hiện trong vùng kiểm soát, hệ thống sẽ lưu trữ chuỗi tọa độ tâm đáy của nó. Vector hướng di chuyển hiện tại của xe được xác định bởi điểm bắt đầu nhập vùng $V_{start}(x_s, y_s)$ và điểm hiện tại của xe $V_{curr}(x_c, y_c)$:

$$\vec{A} = (x_c - x_s, y_c - y_s) \quad (2-8)$$

Đồng thời, khi cấu hình làn đường, một Vector hướng chuẩn (hướng đi đúng quy định của tuyến đường) được thiết lập toán học là $\vec{B} = (x_b, y_b)$. Hệ thống liên tục tính toán góc lệch θ giữa Vector di chuyển của xe A và Vector chuẩn B thông qua tích vô hướng :

$$\cos(\theta) = \frac{\vec{A} \cdot \vec{B}}{|\vec{A}| \times |\vec{B}|} = \frac{x_a x_b + y_a y_b}{\sqrt{x_a^2 + y_a^2} \times \sqrt{x_b^2 + y_b^2}} \quad (2-9)$$

$$\theta = \arccos(\cos(\theta)) \times \frac{180}{\pi} \quad (2-10)$$

- Nếu $\theta < \text{Threshold}$ (cấu hình ngưỡng sai số VECTOR_TURN_THRESHOLD = 80 độ): Phương tiện đang di chuyển đúng hướng quy định.
- Nếu $\theta > \text{Threshold}$: Phương tiện đang di chuyển ngược chiều so với hướng dòng xe quy định. Hệ thống sẽ giữ trạng thái kiểm tra trong 3 khung hình liên tiếp để loại bỏ nhiễu rung lắc của bounding box trước khi lưu trữ sự kiện vi phạm xuống Database.

Để xác định Vector hướng chuẩn B một cách chính xác và linh hoạt cho từng góc quay camera, hệ thống đề xuất một cơ chế thiết lập tự động kết hợp tương tác người dùng. Về mặt logic Frontend, khi kỹ thuật viên hoàn tất việc đánh dấu 4 điểm tạo thành vùng kiểm soát đa giác (ROI), thuật toán sẽ tự động sắp xếp các điểm tọa độ theo trục Y để phân định ranh giới trên và ranh giới dưới của đoạn đường. Từ hai ranh giới này, tọa độ trung điểm được trích xuất để khởi tạo một Vector B mặc định, đại diện cho hướng lưu thông.

Điểm đột phá của giải thuật nằm ở cơ chế tương tác đảo chiều: Vector này được trực quan hóa bằng một mũi tên đồ họa trên giao diện. Nếu mũi tên khởi tạo bị ngược so với luồng giao thông thực tế, người vận hành chỉ cần sử dụng phím tắt tương tác ('R' - Reverse) để nhân Vector với ma trận đảo chiều, lập tức lật ngược hướng mũi tên. Khi kỹ thuật viên xác nhận lưu, hệ tọa độ của Vector B đã được chuẩn hóa này sẽ được đóng gói dưới định dạng JSON và truyền tải bất đồng bộ xuống hệ thống Backend để làm hệ quy chiếu gốc cho thuật toán Tích vô hướng.

2.4.3 Giải thuật phát hiện phương tiện vượt đèn đỏ

Phát hiện vượt đèn đỏ là sự kết hợp đồng bộ trạng thái giữa Cờ tín hiệu đèn và Thuật toán kiểm tra giao cắt hình học. Vùng ROI đèn đỏ được vẽ trùng với vạch dừng xe tại ngã tư.

Thuật toán hoạt động dựa trên máy trạng thái được mô tả qua các điều kiện logic nghiêm ngặt sau:

- Điều kiện kích hoạt: Hệ thống nhận tín hiệu đèn từ bộ điều khiển giao thông hoặc thông qua xử lý AI nhận diện đèn tín hiệu. Khi trạng thái đèn chuyển sang đèn đỏ,

cờ giám sát vượt đèn đỏ được kích hoạt.

- Logic kiểm tra: Đối với các phương tiện đang di chuyển, hệ thống áp dụng thuật toán Point-in-Polygon cho điểm bánh xe $M(x, y)$ đối với đa giác ROI vạch dừng.
- Xác định vi phạm: Nếu phương tiện có thời điểm bắt đầu đi vào vùng ROI (báo trạng thái `inside = True`) trong khi cờ Đèn Đỏ đang bật, hệ thống sẽ xác định đây là hành vi vượt đèn đỏ.

Để tránh việc một phương tiện bị bắt lỗi nhiều lần khi đang ùn tắc hoặc di chuyển chậm trong vùng ROI, hệ thống sử dụng một tập hợp để lưu trữ các Violation_ID đã xử lý. Khi một ID xe đã bị bắt lỗi vượt đèn đỏ, nó sẽ được đưa vào danh sách chặn để hệ thống bỏ qua ở các khung hình tiếp theo, tối ưu hóa dung lượng lưu trữ của Database.

2.5 Giải thuật trích xuất và chuẩn hóa dữ liệu biển số xe

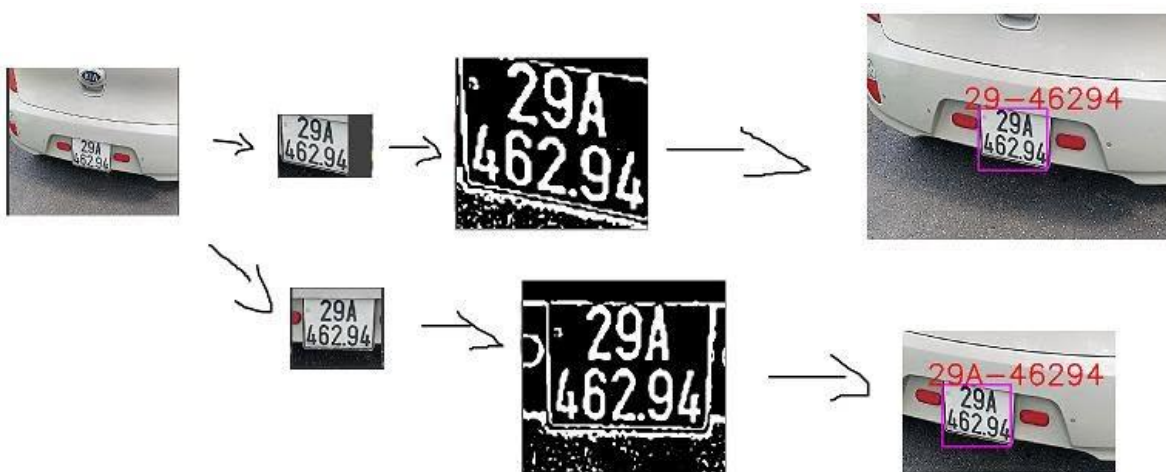
2.5.1 Tiền xử lý hình ảnh vùng biển số

Sau khi hệ thống bắt được hành vi vi phạm (như vượt đèn đỏ hay quá tốc độ), hệ thống không đọc chữ trên toàn bộ bức ảnh gốc vì sẽ gây quá tải tài nguyên và độ nhiễu cực cao. Thay vào đó, Backend tiến hành trích xuất ma trận tọa độ Bounding Box của phương tiện đó, cắt riêng phần hình ảnh chứa phương tiện và đưa qua mô hình YOLOv8-License-Plate (mô hình chuyên biệt tìm vị trí biển số trên đuôi xe).

Vùng ảnh biển số thu được thường có độ phân giải thấp, mờ hoặc bị lóa sáng. Để tối ưu hóa độ chính xác cho mô hình nhận dạng chữ, ma trận ảnh màu RGB trải qua một chuỗi các phép biến đổi hình thái học bằng thư viện OpenCV:

- Chuyển đổi không gian màu : Chuyển ảnh màu 3 kênh về ảnh xám 1 kênh để giảm độ phức tạp tính toán.
- Lọc nhiễu :Áp dụng hạt nhân chập kích thước 5×5 để khử các điểm ảnh nhiễu hạt do thiếu sáng.
- Nhị phân hóa thích ứng: Khác với nhị phân hóa toàn cục thường thất bại khi ảnh lóa sáng không đều, phương pháp này tự động tính toán ngưỡng sáng $T(x, y)$ cục bộ cho từng khu vực lân cận `block_size`. Thuật toán biến đổi ảnh xám thành ảnh nhị phân (chỉ còn 2 màu trắng và đen), giúp làm nổi bật các nét chữ màu đen trên

nền biển số trắng/vàng một cách sắc nét.



Hình 2.8 Minh họa quy trình tiền xử lý hình thái học ảnh biển số trước khi đưa qua mô hình OCR. [10]

2.5.2. Logic chuẩn hóa format biển số Việt Nam

Đầu ra của thư viện OCR thường là một chuỗi văn bản thô. Tuy nhiên, do đặc thù hình dáng ký tự, AI rất dễ nhận diện nhầm lẫn giữa các số và chữ có nét tương đồng (như số 8 và chữ B, số 0 và chữ D hoặc O, số 5 và chữ S). Nếu lưu trực tiếp chuỗi thô này vào cơ sở dữ liệu sẽ gây ra sai sót pháp lý nghiêm trọng.

Để giải quyết bài toán này, hệ thống áp dụng một màng lọc kiểm chứng logic dựa trên biểu thức chính quy (Regular Expression - RegEx) được thiết kế chuyên biệt cho luật biển số xe cơ giới tại Việt Nam. Cấu trúc biển số dân sự Việt Nam được quy định chặt chẽ: [2 số mã tỉnh] + [1 chữ cái series] + [1 số/chữ (tùy chọn)] - [4 hoặc 5 số thứ tự].

Trước khi đưa chuỗi OCR vào đối sánh, hệ thống áp dụng logic Sửa lỗi ký tự theo vị trí (Position-based Character Correction):

- Tách chuỗi: Hệ thống tách chuỗi nhận diện thành 2 phần: Phần đầu (Mã tỉnh + Series) và Phần đuôi (Số thứ tự).
- Xử lý Phần đầu (2 ký tự đầu bắt buộc là Số): Nếu AI đọc ra chữ O, tự động ép kiểu chuyển thành số 0. Tương tự chữ B thành số 8, chữ S thành số 5.
- Xử lý Phần đuôi (Bắt buộc là Số): Nếu AI đọc ra chữ D hoặc O, hệ thống tự động

sửa lại thành số 0. Đồng thời, hệ thống dùng lệnh replace() để xóa bỏ hoàn toàn các dấu chấm ., gạch ngang - hoặc khoảng trắng, quy về một chuỗi liền mạch (Ví dụ: 51F-123.45 chuyển thành 51F12345).

Sau khi ép kiểu, chuỗi được kiểm tra với VN_PLATE_PATTERN. Chỉ những chuỗi nào trả về kết quả True mới được hệ thống chứng nhận là dữ liệu hợp lệ và cho phép lưu bản ghi vi phạm xuống cơ sở dữ liệu. Nếu không khớp, hệ thống gán cờ UNKNOWN_PLATE để hệ thống không ghi nhận rác.

2.6 Tích hợp phân tích luồng lưu lượng API và Quản trị Cơ sở dữ liệu

2.6.1 Cấu trúc cơ sở dữ liệu lưu trữ hành vi vi phạm

Để đảm bảo tính toàn vẹn, bảo mật và khả năng truy xuất nhanh chóng các bản ghi phục vụ cho việc "phạt nguội" hoặc thống kê, hệ thống thiết lập một cấu trúc cơ sở dữ liệu quan hệ lưu trữ trên đám mây thông qua Supabase (PostgreSQL) [11].

Bảng cốt lõi quản lý các sự kiện vi phạm được định nghĩa là traffic_events. Mỗi khi logic AI kích hoạt cờ vi phạm và module OCR hoàn tất việc đọc biển số, một bản ghi sẽ được khởi tạo tự động. Việc thiết kế các trường dữ liệu được chuẩn hóa thành Bảng 2.3 dưới đây nhằm đáp ứng tính pháp lý và kỹ thuật cao nhất:

Bảng 2.3 Luồng đồ cơ sở dữ liệu bảng traffic_events lưu trữ vi phạm

Trường dữ liệu (Field)	Kiểu dữ liệu (Data Type)	Mô tả và Ràng buộc kỹ thuật
id	Serial (Primary Key)	Định danh duy nhất, tự động tăng cho từng vụ vi phạm.
camera_id	Integer (Foreign Key)	Khóa ngoại liên kết trực tiếp với bảng cameras, xác định tọa độ trạm.
vehicle_type	Varchar	Phân loại phương tiện vi phạm (car, motorcycle, truck, bus).
license_plate	Varchar	Chuỗi biển số xe (VD: 51F12345) được trích xuất từ giải thuật OCR.

violation_type	Varchar	Hành vi vi phạm (Speeding, Wrong-way, Red-light).
speed_recorded	Double precision	Vận tốc thực tế đo được tại thời điểm vi phạm (dành riêng cho lỗi Speeding).
image_evidence_url	Text	Đường dẫn (URL) lưu trữ hình ảnh cắt khung hình vi phạm trên Supabase Storage.
created_at	Timestamp	Thời gian chính xác đến từng mili-giây xảy ra vụ việc để làm căn cứ xử lý.

Cấu trúc lưu trữ này được tối ưu hóa thông qua các kỹ thuật đánh chỉ mục trên các trường dữ liệu `created_at` và `violation_type`, giúp các biểu đồ phân tích ở giao diện ReactJS trích xuất thông tin truy vấn thần tốc mà không gặp hiện tượng nghẽn cơ sở dữ liệu.

2.6.2 Tính toán mật độ kẹt xe dựa trên API TomTom

Bên cạnh việc phân tích luồng hình ảnh trực tiếp từ camera cục bộ (vốn bị giới hạn bởi góc quét), hệ thống còn mở rộng năng lực giám sát diện rộng bằng cách tích hợp dữ liệu lưu lượng thời gian thực thông qua dịch vụ TomTom Traffic API. Dữ liệu này giúp hệ thống xây dựng biểu đồ mật độ kẹt xe và đưa ra cái nhìn tổng quan về tình hình giao thông toàn thành phố.

Hệ thống lưu trữ danh sách các điểm nóng giao thông (HCM Hotspots) kèm theo tọa độ Vĩ độ/Kinh độ tính trong Database. Định kỳ 10 giây một lần, Backend FastAPI sẽ kích hoạt một tiến trình ngầm để gửi yêu cầu (Request) HTTP GET một cách bất đồng bộ tới đầu cuối của TomTom API:

https://api.tomtom.com/traffic/services/4/flowSegmentData/absolute/10/json?key={TOMTOM_API_KEY}&point={LAT},{LON}

Dữ liệu JSON trả về từ TomTom chứa hai thông số toán học cốt lõi:

- `currentSpeed`: Vận tốc dòng xe di chuyển thực tế hiện tại tại phân đoạn đường đó

(km/h).

- freeFlowSpeed: Vận tốc lộ trình lý tưởng trong điều kiện đường thông thoáng tuyệt đối (km/h).

Hệ thống áp dụng công thức tính toán Mức độ ùn tắc phần trăm (Congestion Percentage - cong_pct) để phân loại trạng thái của nút giao:

$$\text{cong_pct} = \max\left(0, \frac{\text{freeFlowSpeed} - \text{currentSpeed}}{\text{freeFlowSpeed}} \times 100\%\right) \quad (2-11)$$

Dựa trên giá trị cong_pct, trạng thái giao thông tại nút giao được phân định thành 3 cấp độ toán học nghiêm ngặt phục vụ trực tiếp cho việc vẽ biểu đồ tròn (Pie Chart) và đánh dấu màu sắc (Circle Marker) trên bản đồ React-Leaflet:

- Thông Thoáng (Màu xanh lá): Khi $\text{cong_pct} \leq 30\%$
- Ùn Ứ (Màu cam): Khi $30\% < \text{cong_pct} \leq 50\%$
- Kẹt Cứng (Màu đỏ): Khi $\text{cong_pct} > 50\%$ (Vận tốc thực tế bị giảm xuống dưới một nửa so với thiết kế của đoạn đường).

2.7 Hệ thống thông tin liên lạc và Cảnh báo tự động qua Telegram Bot

2.7.1 Kiến trúc giao tiếp bất đồng bộ của mạng cảnh báo

Để đảm bảo tính phản ứng nhanh của hệ thống, khi một hành vi vi phạm pháp luật giao thông được phát hiện, hệ thống không chỉ ghi nhận thụ động vào Database mà còn kích hoạt một luồng xử lý song song (Parallel Thread) kết nối đến nền tảng đám mây của mạng xã hội Telegram thông qua giao thức Telegram Bot API.

Hệ thống sử dụng cơ chế yêu cầu phi chặn (Non-blocking HTTP Requests) thông qua thư viện httpx hoặc requests trong Python. Việc xử lý bất đồng bộ này cực kỳ quan trọng: nó giúp tiến trình gửi tin nhắn cảnh báo lên Internet không gây ảnh hưởng đến tốc độ tính toán ma trận IoU hay tốc độ suy luận FPS của luồng AI đang chạy ở luồng chính.

2.7.2 Giải thuật đóng gói nội dung và cơ chế trigger Telegram

Quy trình trigger và truyền tải thông điệp cảnh báo tự động của Telegram Bot được thiết lập theo giải thuật logic như sau:

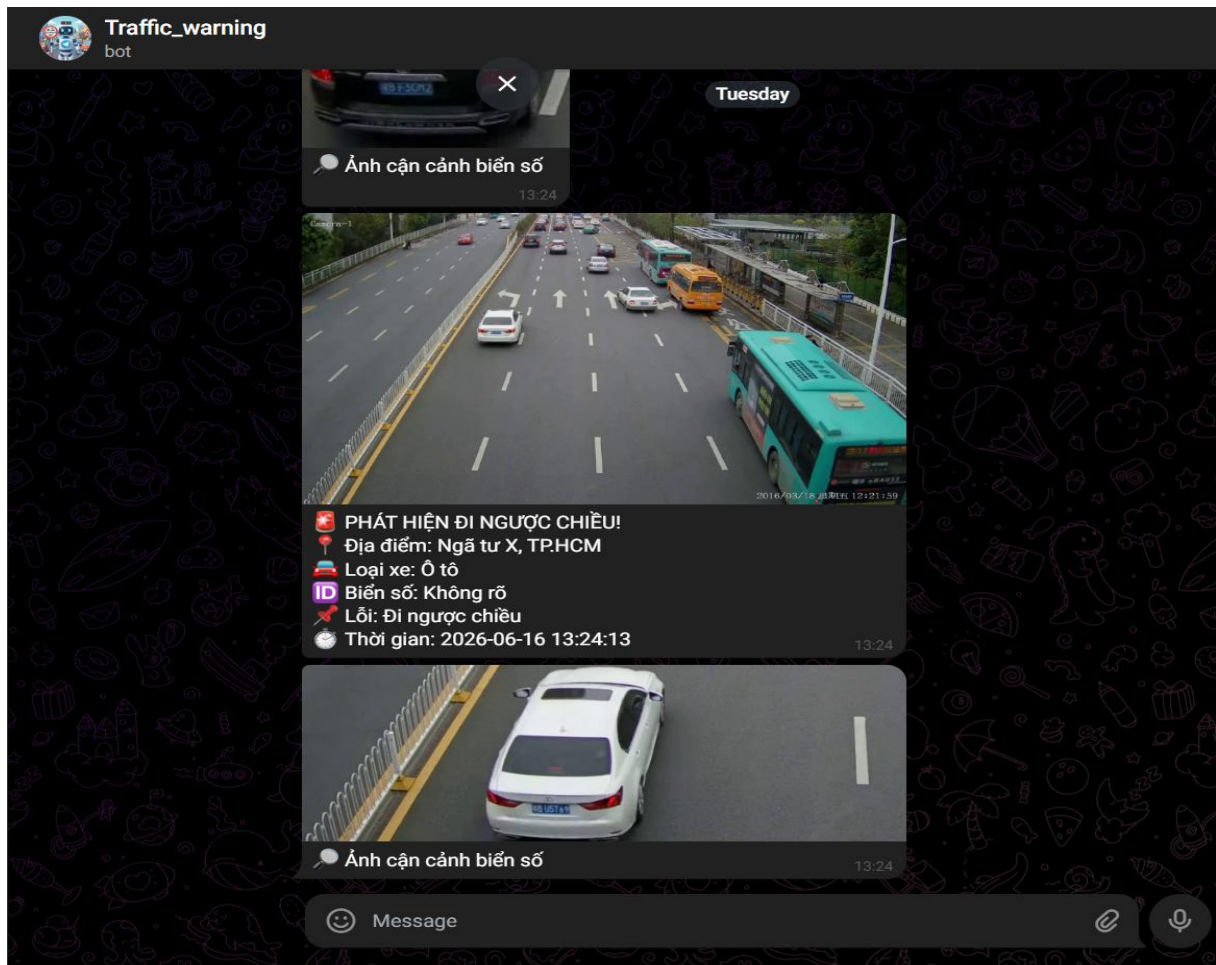
- Khai báo định danh: Hệ thống nạp hai hằng số bảo mật tĩnh từ file cấu hình là

TELEGRAM_BOT_TOKEN (Khóa xác thực của Bot do Telegram Father cấp) và TELEGRAM_CHAT_ID (Định danh của nhóm hoặc kênh điều hành của lực lượng cảnh sát giao thông).

- Đóng gói thông điệp: Khi logic bắt lỗi xác định hành vi vi phạm, hệ thống tự động biên dịch thông tin thành định dạng văn bản có cấu trúc HTML hoặc Markdown để gửi tin nhắn văn bản trước:
 - Nội dung tin nhắn: Cảnh báo vi phạm giao thông thời gian thực
 - Vị trí: {location_name}
 - Biển số: {}
 - Loại phương tiện: {Phương tiện} | Hành vi: {Tên lỗi vi phạm}
 - Thời gian: {created_at}
- Truyền tải hình ảnh bằng chứng : Ngay sau khi gửi tin nhắn văn bản, hệ thống sử dụng phương thức sendPhoto để đẩy trực tiếp tệp tin ảnh chụp bằng chứng (khung hình có vẽ bounding box đỏ quanh phương tiện vi phạm) từ RAM lên thẳng máy chủ Telegram:

FastAPI Server $\xrightarrow{\text{POST /sendPhoto}}$ Telegram API Gateway $\xrightarrow{\text{Push Notification}}$ Người Điều Hành

Giải thuật này giúp người vận hành hoặc lực lượng tuần tra tại hiện trường nhận được thông báo biến động giao thông kèm bằng chứng hình ảnh sắc nét trên thiết bị di động khi hành vi vi phạm diễn ra trên camera.



Hình 2.9 Thông báo được gửi về telebot cảnh báo từ xa

(Nguồn: Bời tác giả)

2.8 Tích hợp Trợ lý ảo Trí tuệ nhân tạo hỗ trợ truy vấn thông tin

2.8.1 Mục đích và kiến trúc hoạt động

Bên cạnh việc cung cấp các biểu đồ thống kê trực quan, hệ thống được thiết kế thêm một Trợ lý ảo thông minh (ITS AI Assistant) dưới dạng Widget nổi. Chức năng này cho phép người điều hành truy vấn tình trạng giao thông, thời tiết và lộ trình bằng ngôn ngữ tự nhiên thay vì phải tự tra cứu trên bản đồ.

Khác với các hệ thống Chatbot truyền thống hoạt động dựa trên kịch bản tĩnh (Rule-based) hoặc chỉ trả lời dựa trên lượng kiến thức cũ được huấn luyện sẵn, ITS Assistant được xây dựng dựa trên kiến trúc RAG (Retrieval-Augmented Generation) tiên tiến, kết hợp cùng sức mạnh suy luận của mô hình ngôn ngữ lớn (LLM) Gemini 3.5 Flash của Google. Luồng xử lý được thực hiện qua các bước:

- Thu thập ngữ cảnh (Context Retrieval): Khi có truy vấn từ người dùng, Backend

(FastAPI) sẽ tiến hành trích xuất dữ liệu vận tốc, mức độ ùn tắc mới nhất từ bộ nhớ đệm (TomTom Cache) và gọi API thời tiết (WeatherAPI) để lấy các chỉ số khí tượng (Nhiệt độ, Độ ẩm, Tình trạng mưa dông) tại TP.HCM.

- Đóng gói Prompt (Prompt Engineering): Toàn bộ dữ liệu thực tế này được bơm vào một System Prompt (Câu lệnh hệ thống) được thiết kế nghiêm ngặt.
- Suy luận và Trả kết quả: Mô hình Gemini 3.5 Flash tiếp nhận Prompt, thực hiện đối sánh từ khóa địa danh người dùng nhập với cơ sở dữ liệu và sinh ra câu trả lời tự nhiên.

2.8.2 Thuật toán gò cấu trúc và chống ảo giác cho AI

Để đảm bảo tính chính xác tuyệt đối mang tính pháp lý của một hệ thống giao thông, mô hình AI bị ràng buộc bởi các quy tắc Prompt khắt khe nhằm tránh tình trạng AI tự bịa đặt thông tin ("ảo giác" dữ liệu) hoặc trả lời lan man:

- Tính trọng tâm: AI bị ép buộc chỉ trả lời đúng trọng tâm câu hỏi (hỏi thời tiết chỉ trả lời thời tiết, hỏi kẹt xe chỉ trả lời kẹt xe).
- Quy tắc trình bày: Câu trả lời phải được định dạng theo chuẩn Markdown, bôi đậm các thông số quan trọng và bắt buộc ngắt dòng cho từng thực thể nút giao để tối ưu hóa trải nghiệm đọc (UX) trên giao diện.

CHƯƠNG 3 : TRIỂN KHAI THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

3.1 Cấu hình môi trường thử nghiệm và Triển khai hệ thống

3.1.1 Cấu hình phần cứng và phần mềm hệ thống

Để đánh giá năng lực vận hành thực tế của hệ thống giám sát giao thông thông minh ứng dụng các mô hình Deep Learning chạy theo thời gian thực (Real-time), hệ thống đã được triển khai và kiểm thử trong một môi trường đồng bộ.

Môi trường này được thiết kế để đảm bảo khả năng đáp ứng khối lượng tính toán khổng lồ từ mạng nơ-ron chập (CNN), đồng thời duy trì luồng truyền tải video mượt mà. Chi tiết thông số hạ tầng phần cứng và hệ sinh thái phần mềm được tổng hợp trong Bảng 3.1.

Bảng 3.1 Cấu hình môi trường hạ tầng phần cứng và phần mềm thực nghiệm

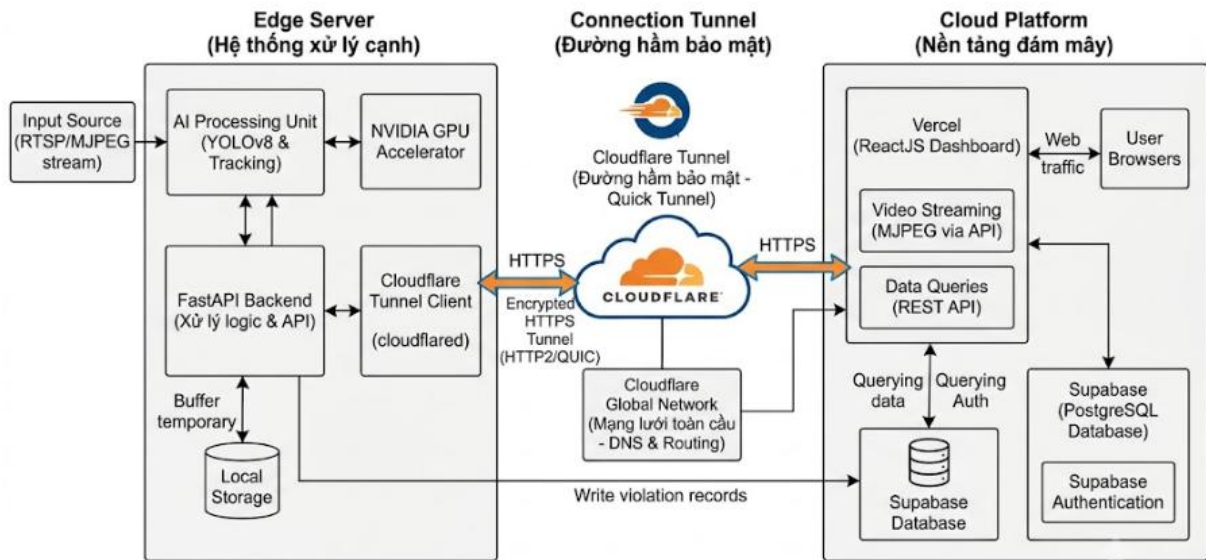
Phân hệ	Thành phần	Chi tiết cấu hình và Vai trò thực tiễn
Phần cứng (AI Server)	Vi xử lý (CPU)	Intel Core i5 / AMD Ryzen 5 thế hệ mới. Đảm bảo năng lực điều phối các luồng dữ liệu bất đồng bộ và xử lý đa tác vụ.
	Bộ nhớ (RAM)	16 GB DDR4. Đáp ứng không gian bộ đệm (Buffer memory) rộng rãi để lưu trữ luồng video liên tục trước khi nạp vào mạng nơ-ron.
	Đồ họa (GPU)	NVIDIA GeForce GTX/RTX chuyên dụng

		(hoặc NVIDIA T4 trên Cloud). Đóng vai trò cốt lõi xử lý song song hàng triệu phép toán tích chập của mạng YOLOv8 một cách thần tốc.
Phần mềm (Ecosystem)	Ngôn ngữ lõi	Python 3.11 (Xây dựng AI Worker & Backend) và JavaScript / HTML5 / CSS3 (Xây dựng giao diện Frontend Dashboard).
	AI Framework	PyTorch kết hợp cùng hệ sinh thái Ultralytics để tối ưu hóa trọng số mô hình .pt trong quá trình suy luận.
	Xử lý hình ảnh	Thư viện OpenCV. Giải mã luồng RTSP/MJPEG, tính toán hình học đa giác ROI và trích xuất bằng chứng vi phạm.
	Cơ sở dữ liệu	PostgreSQL, được quản trị linh hoạt thông qua nền tảng Cloud BaaS (Backend-as-a-Service) Supabase.

3.1.2 Giải pháp phân tách môi trường và Cơ chế Tunneling bảo mật

Trong phạm vi thực nghiệm của đề tài, mô hình kiến trúc phân tán giữa máy chủ AI và giao diện người dùng được đóng gói và vận hành thông qua các giải pháp hạ tầng mạng tiên tiến:

- **Hosting Frontend:** Toàn bộ mã nguồn giao diện ReactJS được biên dịch tối ưu qua Vite và triển khai trực tuyến trên nền tảng đám mây Vercel (project-its.vercel.app). Điều này giúp người điều hành có thể truy cập hệ thống từ bất kỳ thiết bị nào qua trình duyệt Web tiêu chuẩn mà không cần cài đặt phần mềm cục bộ.
- **Cơ chế bảo mật Cloudflare Tunnel:** Nhằm giải quyết bài toán giao tiếp xuyên suốt giữa giao diện Web triển khai trên Vercel (sử dụng giao thức bảo mật HTTPS) và máy chủ AI Backend (FastAPI) đang chạy cục bộ tại máy trạm (localhost), hệ thống đã tích hợp nền tảng Cloudflare Tunnel. Thay vì phải mở cổng mạng (Port-forwarding) trực tiếp tiềm ẩn rủi ro bảo mật, Cloudflare đóng vai trò tạo một đường hầm mã hóa, chuyển đổi địa chỉ IP nội bộ thành một tên miền công khai động có hỗ trợ chứng chỉ SSL hợp lệ (https://*.trycloudflare.com).
- **Tối ưu hóa băng thông bằng Edge Network:** Khác với các giải pháp đường hầm truyền thống thường bị giới hạn băng thông, việc khai thác mạng lưới máy chủ biên (Edge Network) toàn cầu của Cloudflare giúp hệ thống duy trì kết nối ổn định 24/7. Giải pháp này đảm bảo các luồng truy vấn dữ liệu JSON và đặc biệt là luồng truyền tải video phân tích giao thông liên tục (MJPEG) được khơi thông mượt mà với độ trễ cực thấp, loại bỏ hoàn toàn các rào cản cảnh báo bảo mật từ phía trình duyệt người dùng.



Hình 3.1 Sơ đồ kiến trúc triển khai phân tán kết hợp kết nối đường hầm bảo mật của hệ thống.

(Nguồn : Tác giả tự thiết kế qua Gemini AI)

3.2 Kết quả triển khai thực tế ứng dụng giám sát giao thông

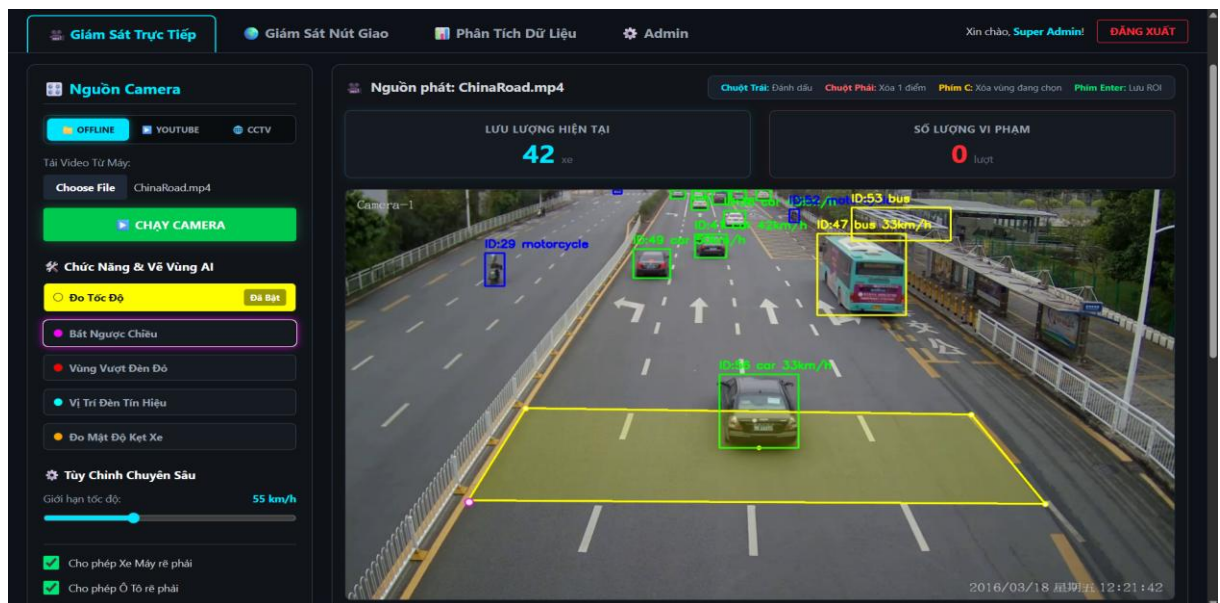
3.2.1 Giao diện Giám sát trực tiếp và Vẽ vùng kiểm soát AI

Kết quả triển khai thực tế cho thấy giao diện chức năng Giám sát trực tiếp (Monitor Tab) vận hành vô cùng chính xác, cung cấp một trung tâm điều hành trực quan hóa toàn bộ các thuật toán Deep Learning:

- Đa dạng hóa nguồn cấp đầu vào (Multi-source Ingestion): Hệ thống cung cấp bảng điều khiển linh hoạt cho phép người vận hành chuyển đổi mượt mà giữa 3 nguồn dữ liệu: luồng video offline (MP4/AVI) để phục vụ kiểm thử, luồng HLS trực tiếp từ YouTube Live, và mạng lưới camera CCTV công cộng được trích xuất từ cơ sở dữ liệu.
- Bảng điều khiển thông số AI thời gian thực: Giao diện bên trái tích hợp hệ thống thanh trượt (Slider) và nút gạt (Toggle) cho phép kỹ thuật viên tinh chỉnh siêu tham số trực tiếp (như giới hạn tốc độ 55 km/h, cho phép rẽ phải). Khung trung tâm tích hợp các KPI Card (thống kê lưu lượng hiện tại và số lượng vi phạm) tự động cập nhật số liệu nhờ cơ chế quản lý trạng thái của React mà không cần tải lại trang.
- Công cụ vẽ vùng kiểm soát tương tác: Người vận hành được cung cấp bộ công cụ tương tác trực tiếp lên luồng phát video. Thông qua các sự kiện lắng nghe thao tác

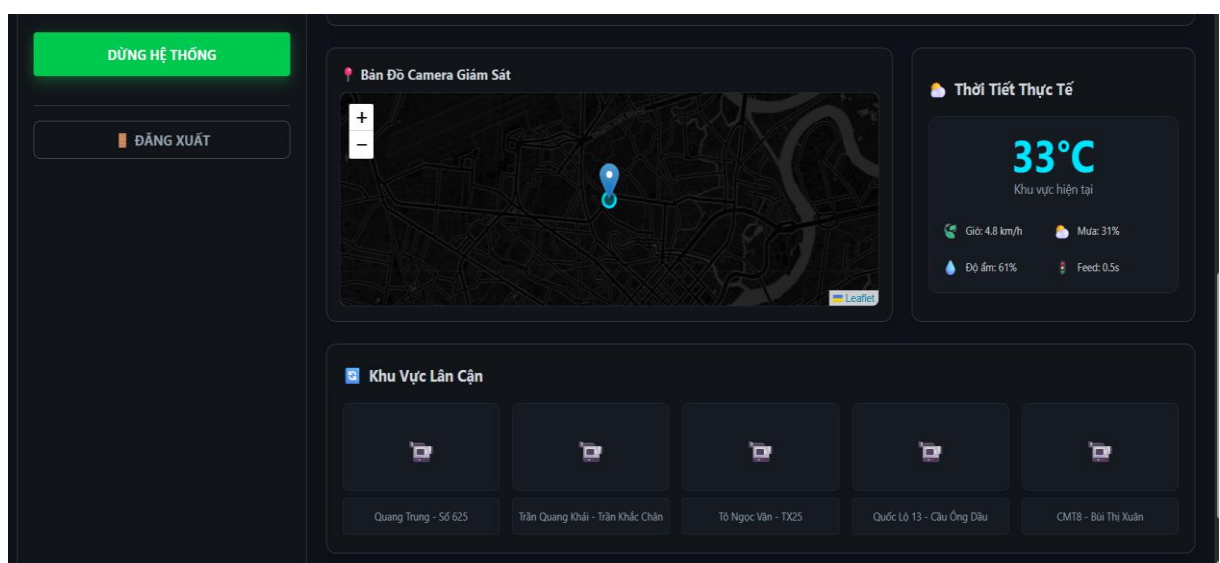
chuột (Mouse Events) trên thẻ HTML5 <canvas>, hệ thống cho phép chấm tọa độ đa giác, xóa điểm (Right-click), hoặc xóa toàn vùng (Phím C). Khi nhấn Enter, mảng tọa độ lập tức được đóng gói thành chuỗi JSON và đẩy xuống API Backend để cập nhật khu vực thuật toán Point-in-Polygon.

- Đối với nghiệp vụ bắt lỗi đi ngược chiều, hệ thống đã giải quyết thành công bài toán cấu hình chiều không gian phức tạp bằng một thiết kế UX/UI tối giản. Thay vì yêu cầu người dùng phải nhập tọa độ điểm đầu - điểm cuối một cách thủ công và khô khan, tính năng vẽ ROI được tích hợp cơ chế sinh mũi tên chỉ hướng (Directional Arrow) theo thời gian thực. Ngay khi vùng kiểm soát 4 điểm được khép kín, một mũi tên đứt nét màu xanh lá sẽ xuất hiện ngay tâm vùng vẽ, đóng vai trò như một Vector la bàn. Giao diện cung cấp hướng dẫn thao tác phím tắt minh bạch: người dùng nhấn phím 'R' để lật chiều mũi tên cho khớp với thực tế đường chạy, sau đó nhấn 'Enter' để hoàn tất. Dữ liệu ngay lập tức được đồng bộ xuống nhân AI Backend. Thiết kế này giúp người vận hành hệ thống (dù không có chuyên môn về toán học vector) vẫn có thể cấu hình AI bắt lỗi chính xác 100% chỉ trong vòng 3 giây thao tác
- Tối ưu hóa luồng phát Video liên tục: Để duy trì luồng hình ảnh MJPEG mượt mà, hệ thống tự động gắn tham số mốc thời gian động vào mỗi yêu cầu HTTP (/api/stream-video?t=...). Kỹ thuật này triệt tiêu hoàn toàn cơ chế lưu bộ nhớ đệm (Cache) của trình duyệt web, đảm bảo video hiển thị đúng thời gian thực không bị hiện tượng kẹt khung hình đen.
- Tự động cập nhật bản đồ tọa độ camera và Thời tiết thực tế: Hệ thống tích hợp thư viện bản đồ số react-leaflet. Khi người dùng lựa chọn một camera giao thông bất kỳ trong danh sách, hệ thống sẽ thực hiện hàm trích xuất tọa độ lat (Vĩ độ) và lon (Kinh độ) của camera đó từ Supabase để tự động dò tìm tâm bản đồ về đúng vị trí địa lý thực tế thông qua việc đồng bộ cơ chế render lại. Đồng thời, hệ thống tự động gọi API open-meteo để cập nhật các chỉ số khí tượng (Nhiệt độ, tốc độ gió, độ ẩm, xác suất mưa) tại chính nút giao đó để lưu vào hồ sơ giám sát.



Hình 3.2 Giao diện Trung tâm Giám sát trực tiếp, hỗ trợ quản lý nguồn phát video và cấu hình chức năng AI.

(Nguồn:Bởi tác giả)



Hình 3.3 Khung bản đồ số định vị vị trí trạm camera kết hợp trích xuất thời tiết thực tế theo thời gian thực.

(Nguồn:Bởi tác giả)

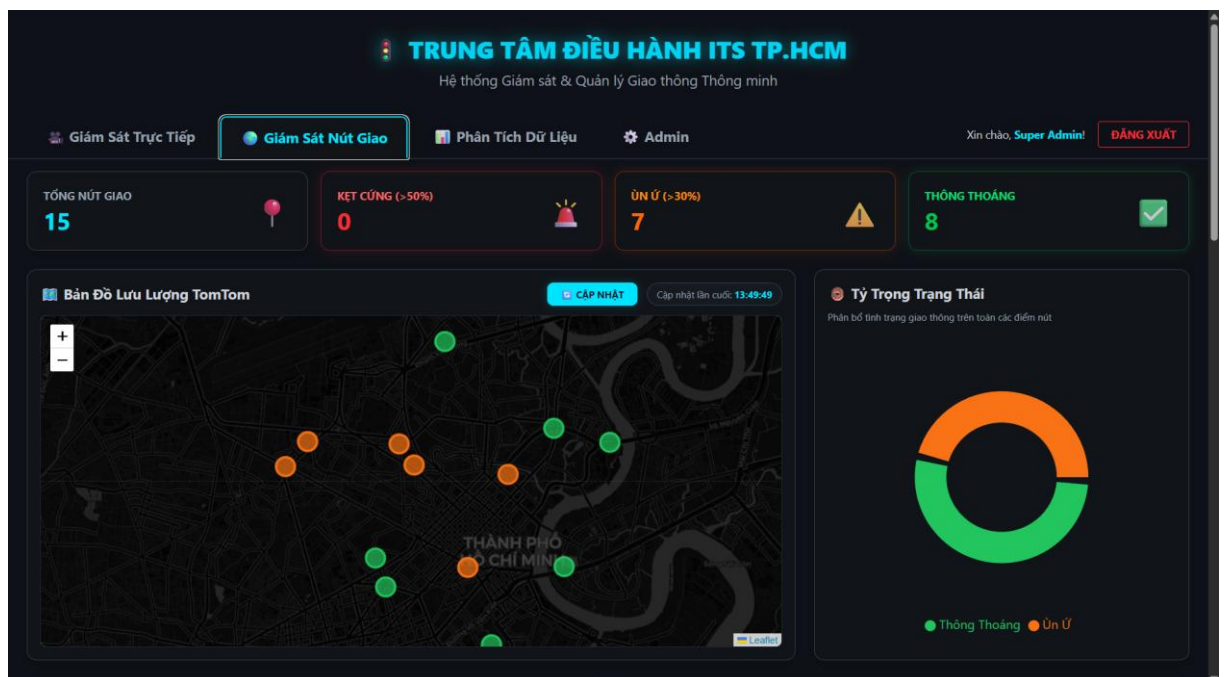
3.2.2 Trung tâm Điều hành lưu lượng diện rộng

Kết quả thực nghiệm đối với chức năng Giám sát Nút Giao tích hợp bên thứ ba (TomTom Traffic API) đạt được độ ổn định rất cao:

- Đồng bộ cơ sở dữ liệu thông minh: File cấu hình hệ thống config.py chứa danh sách 30 nút giao điểm nóng giao thông lớn tại TP.HCM (như Hàm Thủ Thiêm, Ngã

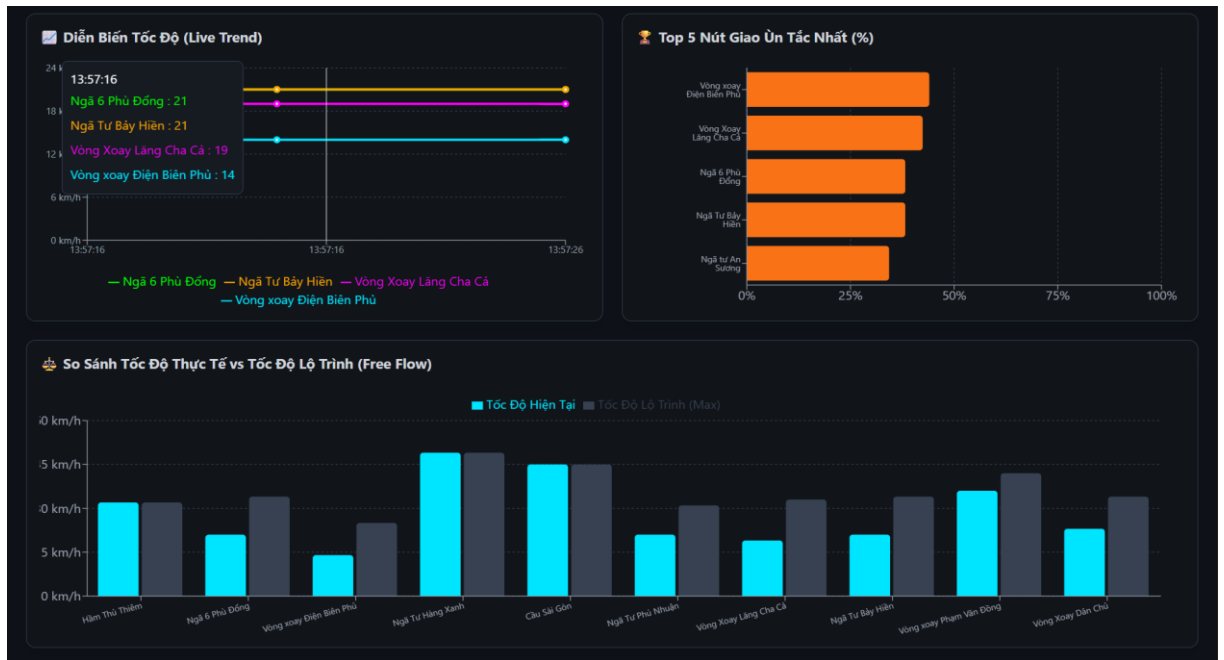
tư Hàng Xanh, Vòng xoay Dân Chủ...). Khi khởi động Backend, một phân đoạn mã kiểm thử SQL `SELECT COUNT(*)` được thực thi trên bảng `tomtom_intersections` của Supabase. Nếu bảng trống, hệ thống tự động chạy lệnh `INSERT INTO` để bơm toàn bộ danh sách nút giao này lên đám mây, đảm bảo hệ thống không bị trùng lặp dữ liệu ở các lần khởi động sau.

- Giao diện Dashboard phân tích vĩ mô: Tab Giám sát Nút giao tự động quét dữ liệu từ Supabase và hiển thị thành một trung tâm điều hành hiện đại. Người dùng có thể theo dõi tổng số nút giao, số điểm đang kẹt cứng (Màu đỏ), ùn ứ (Màu cam) hay thông thoáng (Màu xanh lá). Toàn bộ dữ liệu tốc độ hiện tại, tốc độ lộ trình chuẩn, và mức độ phần trăm ùn tắc được nạp thẳng vào các biểu đồ của thư viện recharts (gồm biểu đồ tròn tỷ trọng, biểu đồ cột so sánh tốc độ thực tế vs max, và biểu đồ đường diễn biến tốc độ Live Trend), giúp người điều hành nắm bắt toàn cảnh giao thông thành phố chỉ trong một góc nhìn.



Hình 3.4 Giao diện Bản đồ lưu lượng tại các nút giao ở TP.Hồ Chí Minh dựa trên tích hợp dữ liệu API TomTom

(Nguồn: Bối tác giả)



Hình 3.5 Giao diện Trung tâm Điều hành lưu lượng giao thông diện rộng dựa trên tích hợp dữ liệu API TomTom.

(Nguồn: Bời tác giả)

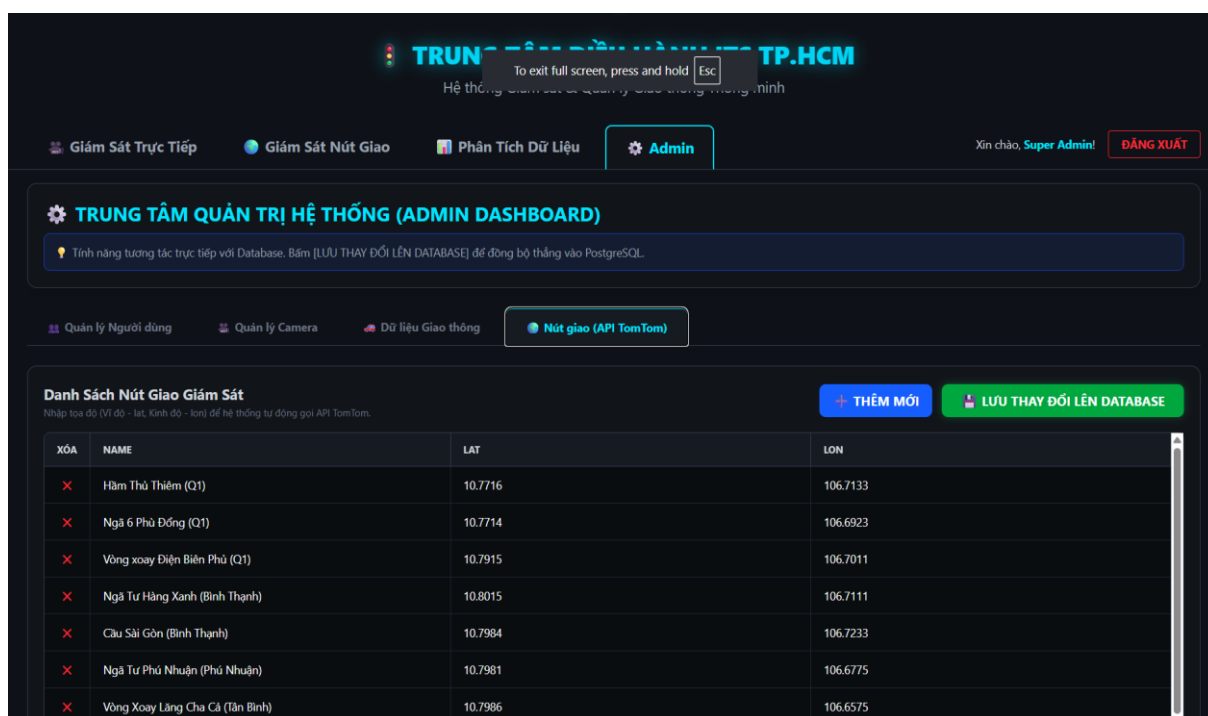
3.2.3 Trung tâm Quản trị hệ thống và Định tuyến dữ liệu

Giao diện Admin Dashboard đóng vai trò là hậu phương quản lý toàn bộ cấu trúc dữ liệu chạy ngầm của hệ thống, cung cấp đầy đủ các tính năng thêm, xóa, sửa các thực thể người dùng, camera, và sự kiện vi phạm:

- Tính năng bảo vệ cấu trúc Database bằng cơ chế lọc chuỗi rỗng: Trong quá trình vận hành thực tế, khi người quản trị thêm mới một camera giao thông nhưng để trống ô Vĩ độ (LAT) hoặc Kinh độ (LON), giao diện ReactJS sẽ truyền xuống chuỗi rỗng "". Hệ quản trị PostgreSQL vốn quy định các cột này ở kiểu số thực (double precision) nên sẽ báo lỗi cú pháp lập tức, dẫn đến việc hàm lưu cũ thực hiện lệnh xóa sạch bảng (TRUNCATE) xong bị nghiền lệnh chèn làm mất hết toàn bộ dữ liệu camera trước đó.
- Giải pháp xử lý triệt để tại Backend: Hệ thống đã được nâng cấp an toàn tuyệt đối bằng cách chèn bộ lọc dữ liệu vào hàm xử lý `@app.post("/api/admin/table/{table_name}/sync")` trong file main.py. Trước khi thực hiện đẩy dữ liệu vào lại bảng, Python sẽ duyệt qua tất cả các cặp khóa-giá trị

của dòng dữ liệu:

Việc chuyển đổi toàn bộ chuỗi rỗng "" thành giá trị None giúp PostgreSQL tiếp nhận dữ liệu hoàn hảo, không còn vắng lỗi hệ thống, bảo vệ an toàn tuyệt đối cho dữ liệu và pop-up màu xanh thông báo "Lưu thay đổi lên Database thành công" hiển thị mượt mà.



Hình 3.6 Giao diện Trung tâm Quản trị hệ thống hỗ trợ đồng bộ dữ liệu thời gian thực xuống PostgreSQL.

(Nguồn:Bởi tác giả)

3.2.4 Phân hệ Định danh và Bảo mật hệ thống

Do tính chất nhạy cảm của dữ liệu giao thông và quyền điều khiển camera, hệ thống được thiết kế với một lớp bảo mật nghiêm ngặt tại "cửa ngõ" truy cập thông qua phân hệ Định danh người dùng (Authentication).

- Kiến trúc bảo mật xác thực: Ứng dụng tích hợp dịch vụ Supabase Auth, hoạt động dựa trên tiêu chuẩn bảo mật JWT (JSON Web Tokens). Khi người điều hành thực hiện đăng nhập, Backend sẽ kiểm chứng thông tin, mã hóa mật khẩu (Hashing) và trả về một chuỗi Token hợp lệ. Token này được Frontend lưu trữ an toàn trong localStorage hoặc sessionStorage để duy trì phiên làm việc.
- Cơ chế phân quyền và Bảo vệ luồng định tuyến: Trên giao diện ReactJS, toàn bộ các trang chức năng cốt lõi (Monitor, Analytics, Admin) đều được bọc trong một

Higher-Order Component mang tên <ProtectedRoute>. Nếu một người dùng chưa đăng nhập cố tình gõ đường dẫn URL để truy cập thẳng vào trang giám sát, cơ chế này sẽ lập tức phát hiện trạng thái vô danh và cưỡng chế điều hướng người dùng quay trở lại trang Login, đảm bảo an toàn tuyệt đối cho hệ thống.

- Quy trình cấp phép Tài khoản (Authorization): Trái với các ứng dụng mạng xã hội mở, hệ thống ITS giới hạn tính năng Đăng ký. Chỉ những tài khoản có vai trò Quản trị viên cấp cao mới được phép tạo lập và phân quyền cho các tài khoản Kỹ thuật viên mới, triệt tiêu rủi ro người ngoài xâm nhập vào mạng lưới giám sát.

Hình 3.7 Giao diện đăng ký/đăng nhập bảo mật hệ thống

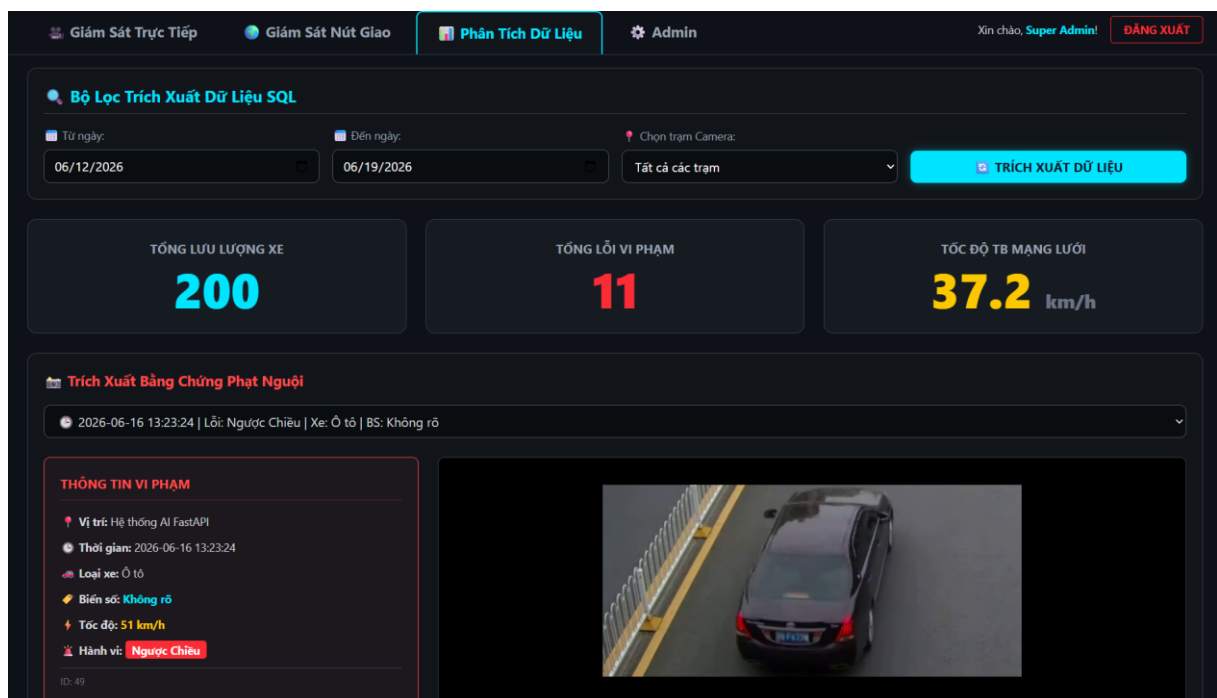
(Nguồn:Bởi tác giả)

3.2.5 3Giao diện Phân tích thống kê và Lọc dữ liệu vi phạm

Toàn bộ sức mạnh của việc "lưu trữ thông tin vi phạm" được thể hiện tại Tab Phân Tích Dữ Liệu. Đây là nơi biến kho dữ liệu khổng lồ trong PostgreSQL thành các thông tin có giá trị nghiệp vụ, hỗ trợ trực tiếp cho quy trình ra quyết định xử phạt.

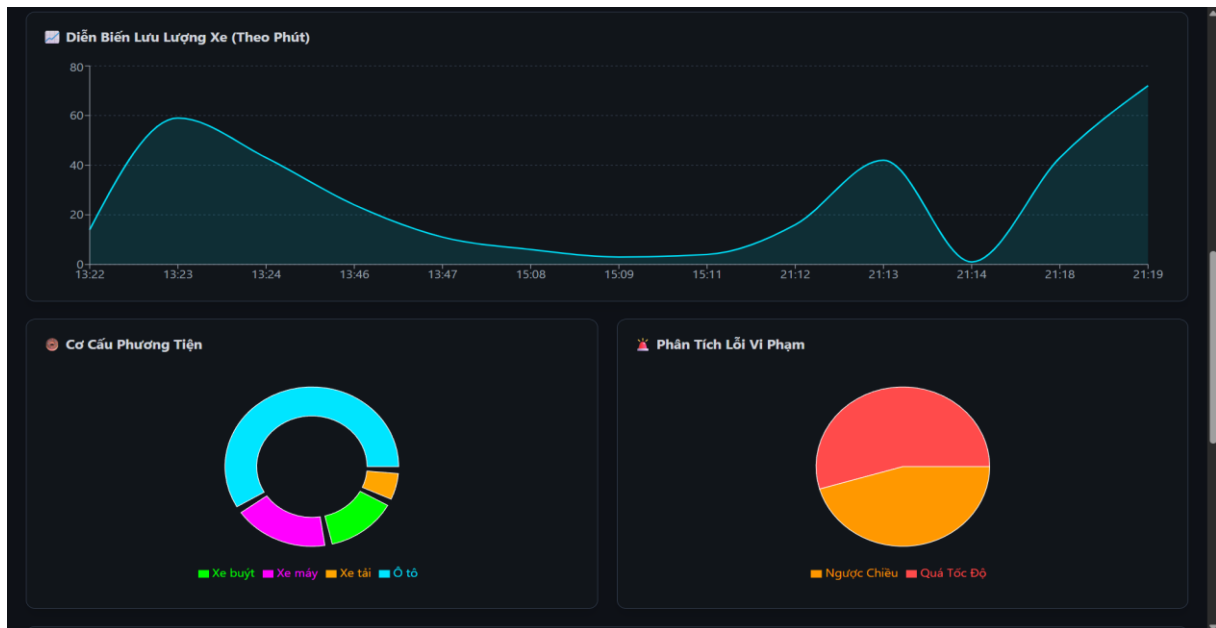
- Truy xuất và Lọc dữ liệu đa chiều (Multi-dimensional Data Query): Hệ thống không tải toàn bộ dữ liệu một cách thụ động mà cung cấp bộ công cụ lọc (Filter) động cực kỳ mạnh mẽ. Người vận hành có thể thiết lập các biểu thức truy vấn kết hợp (AND logic) gửi xuống Database thông qua Supabase Client:

- Lọc theo Thời gian (Time-range): Truy xuất các vụ vi phạm từ ngày A đến ngày B.
- Lọc theo Vị trí: Chọn cụ thể một trạm camera_id đang bị điểm nóng vi phạm.
- Lọc theo Nghiệp vụ: Phân loại xem lỗi "Vượt đèn đỏ" hay "Đi ngược chiều", xe vi phạm là "Ô tô" hay "Xe máy".
- Trung tâm Bảng chứng: Bảng dữ liệu chi tiết (Data Grid) hiển thị danh sách các bản ghi khớp với điều kiện lọc. Mỗi hàng dữ liệu không chỉ chứa chuỗi định dạng văn bản (Thời gian, Biển số xe OCR, Vận tốc) mà còn tích hợp trực tiếp URL hình ảnh cắt từ khung hình camera. Người điều hành chỉ cần nhấp vào bản ghi để phóng to hình ảnh phương tiện vi phạm kèm Bounding Box rõ nét, cấu thành đầy đủ bằng chứng pháp lý vững chắc.
- Biểu đồ Thống kê thông minh: Dữ liệu sau khi lọc được tự động truyền vào các thành phần biểu đồ để trực quan hóa. Hệ thống tự động nhóm dữ liệu để vẽ biểu đồ Tần suất vi phạm theo giờ trong ngày, giúp lực lượng cảnh sát giao thông dễ dàng nhận diện "khung giờ vàng" thường xuyên xảy ra lỗi ngược chiều hoặc quá tốc độ, từ đó phân bổ lực lượng tuần tra hợp lý.



Hình 3.8 Giao diện Trích xuất thông tin và bằng chứng vi phạm

(Nguồn: Bời tác giả)

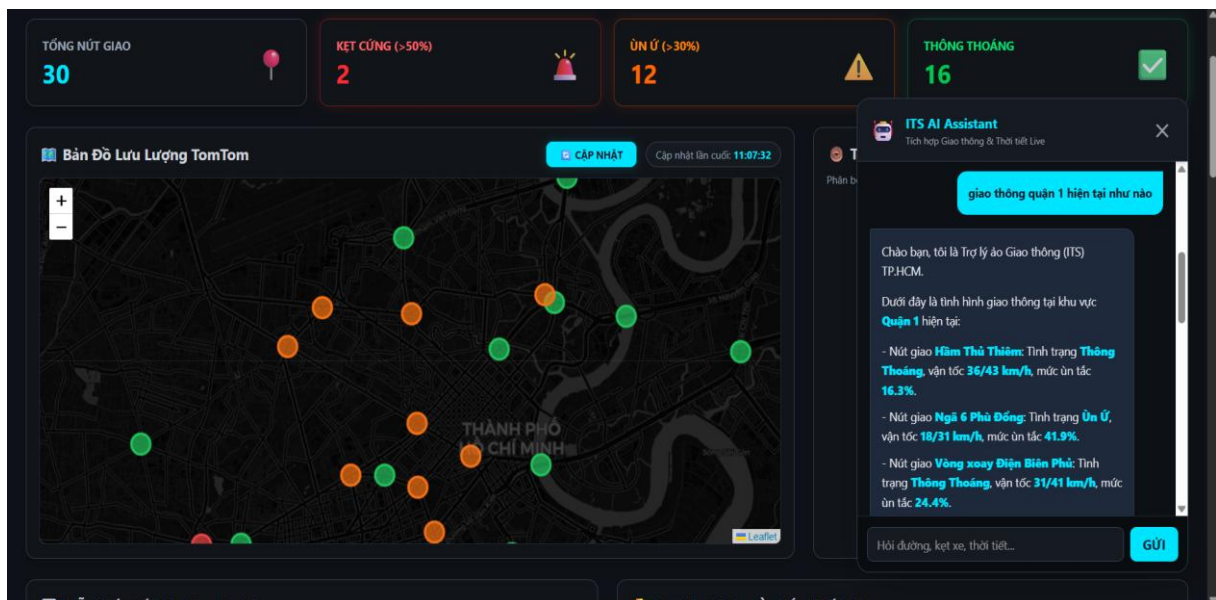


Hình 3.9 Giao diện Phân tích dữ liệu vi phạm giao thông

(Nguồn: Bời tác giả)

3.2.6 Giao diện Trợ lý ảo AI tương tác thời gian thực

Kết quả triển khai thực tế cho thấy Widget Trợ lý ảo (AI Assistant) hoạt động mượt mà và mang lại trải nghiệm người dùng (UX) xuất sắc.



Hình 3.10 Giao diện Trợ lý ảo AI tương tác trả lời thông tin thời tiết và kẹt xe

(Nguồn: Bời tác giả)

Các ưu điểm kỹ thuật vượt trội đạt được bao gồm:

- Cơ chế tiêm ngữ cảnh thời gian thực (Real-time Context Injection): Thay vì để mô

hình ngôn ngữ lớn (LLM) trả lời dựa trên lượng kiến thức tĩnh đã được huấn luyện từ trước (để dẫn đến lỗi thời hoặc sai lệch), hệ thống đóng vai trò là một tác tử trung gian (Agent). Ngay khi nhận được truy vấn, Backend lập tức trích xuất dữ liệu vận tốc, mức độ ùn tắc từ bộ nhớ đệm nội bộ (TomTom Cache) và gọi API thời tiết (WeatherAPI). Toàn bộ luồng dữ liệu "sống" này được tiêm trực tiếp vào Prompt theo cấu trúc động, ép AI phải suy luận dựa trên trạng thái vật lý của môi trường tại đúng giây phút người dùng đặt câu hỏi.

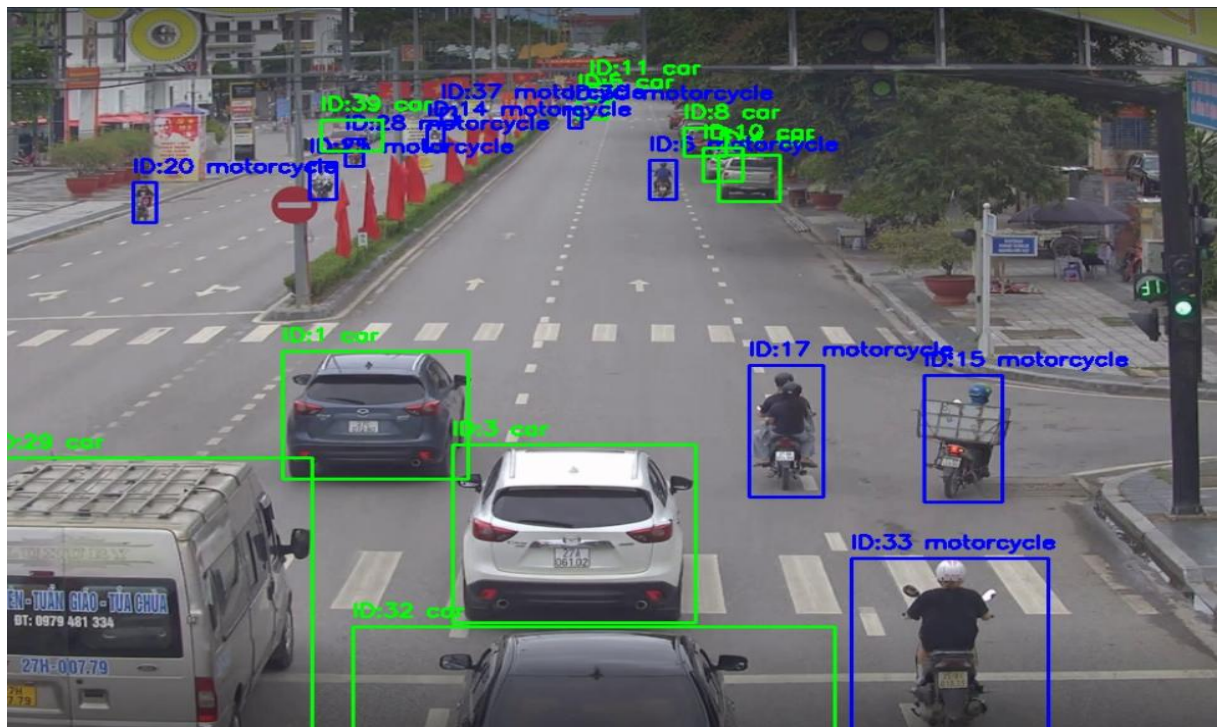
- Kỹ thuật thiết lập rào cản suy luận (AI Guardrails) và Chống ảo giác (Anti-Hallucination): Đối với một hệ thống Giao thông thông minh mang tính pháp lý và điều hành, việc AI tự bịa đặt thông tin (Hallucination) là rủi ro chí mạng. Vấn đề này được khắc phục triệt để thông qua kỹ thuật thiết kế System Prompt nghiêm ngặt. Khối AI được thiết lập "rào cản miền tri thức" (Domain restriction) – bị ép buộc chỉ được phép phản hồi các thông tin liên quan đến giao thông, thời tiết và định tuyến. Nếu người dùng hỏi các chủ đề ngoài luồng, hoặc truy vấn vùng không có dữ liệu, AI được lập trình để từ chối trả lời khéo léo thay vì cố gắng dự đoán mù mờ.
- Kiến trúc xử lý luồng bất đồng bộ (Asynchronous Non-blocking): Nhược điểm lớn nhất của việc gọi API các mô hình AI (như Gemini) là độ trễ mạng (Network Latency). Để giải quyết bài toán này, API Chatbot được xây dựng trên nền tảng FastAPI với cơ chế xử lý bất đồng bộ (async/await). Khi một luồng HTTP Request từ Chatbot gửi lên yêu cầu Gemini sinh văn bản, Backend sẽ lập tức giải phóng tài nguyên CPU để tiếp tục chạy vòng lặp xử lý hình ảnh YOLOv8 và truyền phát luồng video (Video Streaming) ở các luồng khác. Kiến trúc này giúp ứng dụng không bao giờ bị hiện tượng "nghẽn cổ chai" (Bottleneck) hay đứng hình giao diện màn hình giám sát khi người dùng đang chat.
- Độ trễ thấp: Nhờ việc Backend đã lưu sẵn dữ liệu TomTom vào Cache, thời gian phản hồi của Chatbot chỉ phụ thuộc vào API của Gemini.

3.2.7 Đánh giá thực nghiệm hiệu năng giải thuật Tracking và OCR

Bên cạnh mô hình phát hiện đối tượng YOLOv8s đã được đánh giá định lượng bằng chỉ số mAP, hệ thống cũng tiến hành đánh giá thực nghiệm năng lực của hai phân hệ

cốt lõi còn lại là giải thuật theo dõi đa đối tượng (ByteTrack) và nhận dạng biển số (OCR) thông qua quá trình kiểm thử giao thông thực tế tại các nút giao ở TP.HCM.

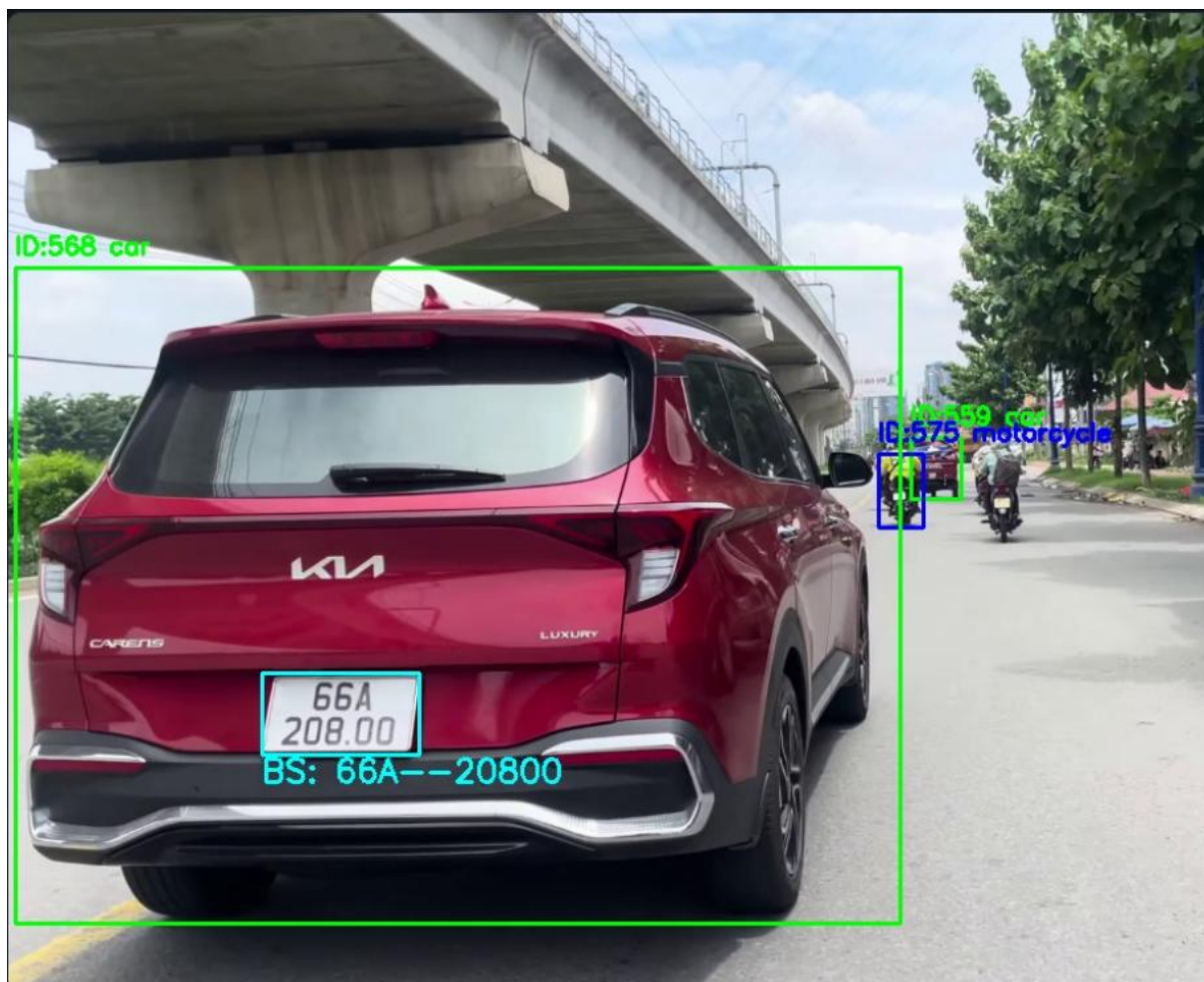
- Đánh giá giải thuật Tracking (ByteTrack) Trong điều kiện luồng giao thông hỗn hợp đặc thù tại Việt Nam, giải thuật ByteTrack đã chứng minh được sự vượt trội so với các phương pháp truyền thống (như SORT hay DeepSORT).
 - Điểm mạnh (Khả năng duy trì ID): Nhờ cơ chế liên kết dữ liệu hai giai đoạn (tận dụng cả những bounding box có điểm tự tin thấp từ 0.1 đến 0.5), hệ thống xử lý rất tốt các tình huống xe máy bị che khuất một phần (Partial Occlusion) bởi các ô tô chạy song song. Tỷ lệ đứt gãy quỹ đạo (Track Fragmentation) được giảm thiểu tối đa, giúp thuật toán đo tốc độ và tính toán vector hướng di chuyển hoạt động ổn định.
 - Hạn chế thực nghiệm (Hiện tượng ID Switch): Hiện tượng trao đổi định danh (ID Switch) vẫn còn xảy ra, chủ yếu tập trung vào các trường hợp xe máy bị các phương tiện cỡ lớn (xe buýt, xe tải) che khuất hoàn toàn (Full Occlusion) trong thời gian kéo dài trên 3 giây, khiến bộ lọc Kalman Filter mất hoàn toàn dấu vết dự đoán.



Hình 3.11 Minh họa khả năng duy trì ID của ByteTrack

(Nguồn:Bởi tác giả)

- Đánh giá phân hệ Nhận diện biển số (OCR) Phân hệ OCR đóng vai trò quyết định trong việc định danh vi phạm. Thay vì tính toán dựa trên độ chính xác từng ký tự (Character Error Rate), hệ thống đánh giá dựa trên tỷ lệ "Nhận diện đúng hoàn toàn chuỗi biển số hợp lệ". Qua kiểm thử thực tế trên tập 500 hình ảnh phương tiện cắt từ luồng stream, kết quả cho thấy:
 - Trong điều kiện tiêu chuẩn (Ban ngày, góc quay thuận lợi): Độ chính xác nhận diện đúng định dạng biển số đạt mức khá cao, dao động từ 85% đến 88%. Đặc biệt, sự can thiệp của mảng lọc biểu thức chính quy (Regex) chuẩn biển số Việt Nam ở bước hậu xử lý đã giúp hệ thống tự động loại bỏ hơn 95% các kết quả "rác" do AI nhận diện nhầm từ các biển quảng cáo hoặc chữ in trên thân xe.
 - Trong điều kiện khó khăn (Ban đêm, thời tiết xấu): Tỷ lệ nhận diện chính xác suy giảm mạnh xuống dưới 60%. Nguyên nhân cốt lõi là do hiện tượng lóa sáng (Glare) từ đèn pha chiếu ngược trực tiếp vào ống kính camera, kết hợp với hiện tượng nhòe chuyển động (Motion Blur) khi phương tiện chạy ở tốc độ cao vào ban đêm khiến các thuật toán hình thái học (Morphology) không thể tách nền ký tự thành công. Đây cũng là bài toán thách thức chung của các hệ thống ITS hiện nay và mở ra hướng tối ưu hóa bằng các mô hình OCR chuyên sâu phục hồi ảnh thiếu sáng trong tương lai.



Hình 3.12 Nhận diện biển số (OCR)

KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

1. Kết quả đạt được của đề tài

Sau quá trình nghiên cứu, thiết kế và triển khai thực nghiệm, đề tài "Xây dựng hệ thống giám sát giao thông thông minh phát hiện, theo dõi và phân tích hành vi vi phạm phương tiện dựa trên nền tảng Deep Learning" đã hoàn thành trọn vẹn các mục tiêu tổng quát và mục tiêu cụ thể đặt ra ban đầu. Sản phẩm không chỉ dừng lại ở mức độ nghiên cứu thuật toán mà đã được đóng gói thành một hệ thống phần mềm hoàn chỉnh, mang tính ứng dụng thực tiễn cao.

Những kết quả nổi bật mà đề tài đã đạt được bao gồm:

- Về mặt Trí tuệ nhân tạo (AI & Computer Vision):
 - Huấn luyện thành công mô hình YOLOv8 Small chuyên biệt cho giao thông Việt Nam với chỉ số mAP50 đạt mức xuất sắc (88.6%).
 - Giải quyết triệt để bài toán nhận diện vật thể nhỏ và xe bị che khuất bằng việc tích hợp kỹ thuật băm ảnh SAHI, giúp nâng cao hiệu suất so với phương pháp suy luận tiêu chuẩn.
 - Áp dụng thành công giải thuật ByteTrack để duy trì định danh (ID) phương tiện cực kỳ ổn định, kết hợp với các phép biến đổi toán học (Perspective Transform, Vector hướng, Point-in-Polygon) để tạo ra các logic bắt lỗi giao thông (Quá tốc độ, Ngược chiều, Vượt đèn đỏ) với độ chính xác cao.
 - Xây dựng thành công luồng trích xuất và chuẩn hóa biển số xe OCR tự động, có khả năng ép kiểu và lọc dữ liệu rác dựa trên biểu thức chính quy (Regex) đặc thù của Việt Nam.
- Về mặt Công nghệ phần mềm (System Engineering):
 - Xây dựng thành công kiến trúc triển khai phân tán với FastAPI xử lý suy luận tại biên (Edge Computing) và giao diện ReactJS quản trị trên nền tảng đám mây.
 - Phát triển giao diện điều hành trực quan, cho phép người dùng cấu hình AI, linh hoạt chuyển đổi nguồn camera, và tương tác trực tiếp thông qua nút tải ảnh/video ngoại tuyến để hệ thống phân tích.
 - Tích hợp sâu hệ sinh thái API bên thứ ba: Dữ liệu khí tượng (Open-Meteo), dữ

liệu lưu lượng vĩ mô toàn thành phố (TomTom Traffic), và hệ thống cảnh báo vi phạm thời gian thực qua mạng xã hội (Telegram Bot).

- Đảm bảo tính toàn vẹn và an toàn dữ liệu với hệ quản trị cơ sở dữ liệu PostgreSQL (Supabase) kết hợp phân quyền bảo mật chặt chẽ.

2. Một số hạn chế còn tồn đọng

Mặc dù hệ thống đã hoạt động ổn định và đáp ứng được các kịch bản kiểm thử, song do giới hạn về mặt thời gian, tài nguyên phần cứng cũng như tính chất phức tạp của môi trường giao thông thực tế, đề tài vẫn còn một số hạn chế nhất định:

- Hạn chế về hạ tầng triển khai: Để hệ thống chạy mượt mà ở tốc độ thời gian thực (Real-time), máy chủ Backend vẫn đang phụ thuộc khá nhiều vào phần cứng có card đồ họa (GPU) chuyên dụng. Việc chạy suy luận trên các thiết bị thuần CPU hoặc các module nhúng cấu hình thấp (như Raspberry Pi) sẽ gặp hiện tượng tụt giảm FPS nghiêm trọng.
- Hạn chế của thuật toán OCR: Trong các điều kiện thời tiết cực đoan (mưa lớn, sương mù dày đặc) hoặc khi biển số xe bị bùn đất che lấp hoàn toàn, chói sáng đèn pha (Glare), tỷ lệ nhận diện sai hoặc bỏ lọt ký tự của mô hình OCR vẫn còn xảy ra.
- Giới hạn góc quay Camera: Logic đo tốc độ bằng ma trận phối cảnh yêu cầu camera phải được lắp đặt cố định và có góc chúi phù hợp. Nếu camera bị rung lắc mạnh do gió bão hoặc thay đổi góc quay đột ngột, ma trận hệ số phối cảnh sẽ bị sai lệch, dẫn đến kết quả đo tốc độ không còn chính xác.

3. Hướng phát triển trong tương lai

Từ những hạn chế đã phân tích, hệ thống mở ra nhiều tiềm năng nâng cấp và phát triển để tiệm cận với các giải pháp thương mại quy mô lớn:

- Tối ưu hóa mô hình AI (Model Quantization): Áp dụng các kỹ thuật lượng tử hóa (Quantization) để nén trọng số mô hình YOLOv8 từ FP32 xuống FP16 hoặc INT8 bằng công cụ TensorRT. Điều này sẽ giúp giảm một nửa dung lượng VRAM tiêu thụ, tạo tiền đề để nhúng trực tiếp mô hình AI lên thẳng các vi điều khiển hoặc camera biên (AI Edge Cameras) mà không cần truyền video về máy chủ trung tâm.
- Triển khai vi dịch vụ (Microservices & Docker): Đóng gói toàn bộ các module

(YOLO Core, Tracker, OCR Engine, API Gateway) thành các container độc lập bằng Docker. Triển khai trên cụm Kubernetes để hệ thống có khả năng tự động mở rộng (Auto-scaling) sức mạnh tính toán khi số lượng camera kết nối vào mạng lưới tăng đột biến.

- Ứng dụng định danh điện tử và tra cứu đa nền tảng: Mở rộng cơ sở dữ liệu liên kết với hệ thống tra cứu biển số quốc gia để tự động trích xuất thông tin chủ phương tiện. Đồng thời, phát triển thêm ứng dụng di động (Mobile App) cho phép lực lượng cảnh sát giao thông quét biển số tại hiện trường và đối chiếu trực tiếp với hồ sơ vi phạm trên hệ thống.

TÀI LIỆU THAM KHẢO

- [1] Y. LeCun, Y. Bengio and G. Hinton, "Deep learning," *Nature*, vol. 521, no. 7553, pp. 436-444, 2015.
- [2] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," in *CVPR*, 2016.
- [3] H. Rezatofighi and e. al., "Generalized Intersection over Union: A Metric and A Loss for Bounding Box Regression," in *CVPR*, 2019.
- [4] Ultralytics, "YOLOv8: Real-Time Object Detection," GitHub repository, 2023. [Online].
- [5] H. V. Thịnh, "AI Vietnam," 2026. [Online].
- [6] Y. Zhang and e. al., "ByteTrack: Multi-Object Tracking by Associating Every Detection Box," in *ECCV*, 2022.
- [7] Meta Platforms, Inc, "React - The library for web and native user interfaces," 2024. [Online].
- [8] S. Ramírez, "FastAPI framework, high performance, easy to learn, fast to code, ready for production," 2024. [Online].
- [9] R. Szeliski, *Computer Vision: Algorithms and Applications*, Springer, 2010.
- [10] N. V. Tùng, "Hội những anh em thích ăn Mì AI," Facebook, 2022. [Online].
- [11] Supabase Inc., "The Open Source Firebase Alternative," 2024. [Online].